

+
◦ • [COMSE6998-015] Fall
2025

Introduction to Deep
Learning and LLM based
Generative AI Systems
Parijat Dube and Chen Wang

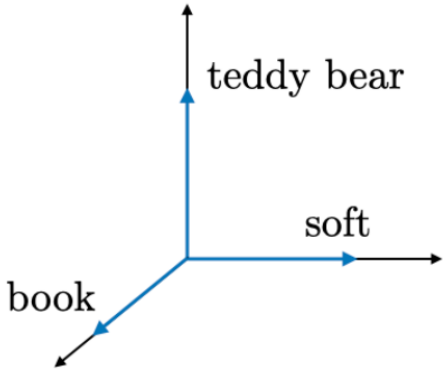
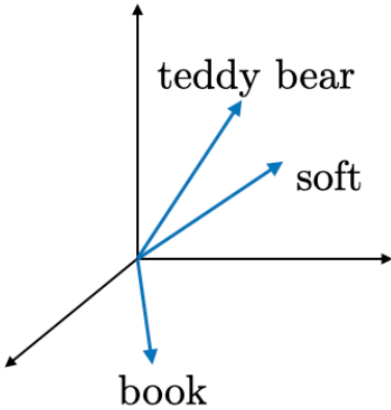
Lecture 2
09/18/26



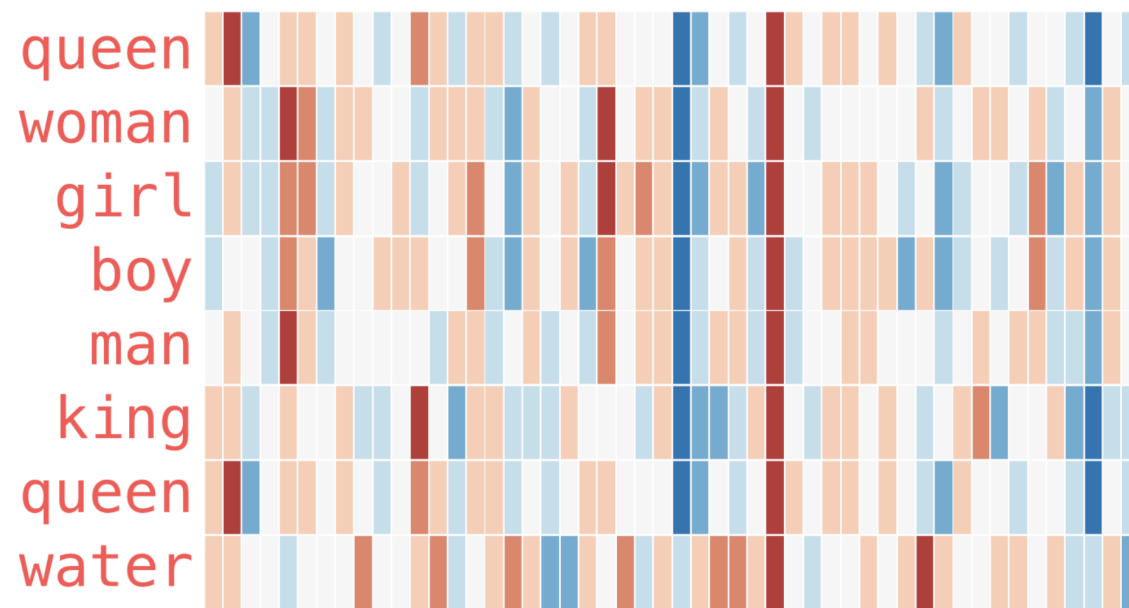
Agenda

- Word embeddings
- Attention mechanism
- Transformer architecture
- Mixture of Experts
- Encoder only vs Decoder only models

Word Embeddings

1-hot representation	Word embedding
 A 3D coordinate system with three axes. Three blue arrows originate from the origin: one points along the positive vertical axis and is labeled 'teddy bear', one points along the positive horizontal axis and is labeled 'soft', and one points along the positive diagonal axis and is labeled 'book'.	 A 3D coordinate system with three axes. Three blue arrows originate from the origin: one points into the upper-right quadrant and is labeled 'teddy bear', one points into the upper-left quadrant and is labeled 'soft', and one points into the lower-left quadrant and is labeled 'book'.
• Noted o_w • Naive approach, no similarity information	• Noted e_w • Takes into account words similarity

Word2Vec



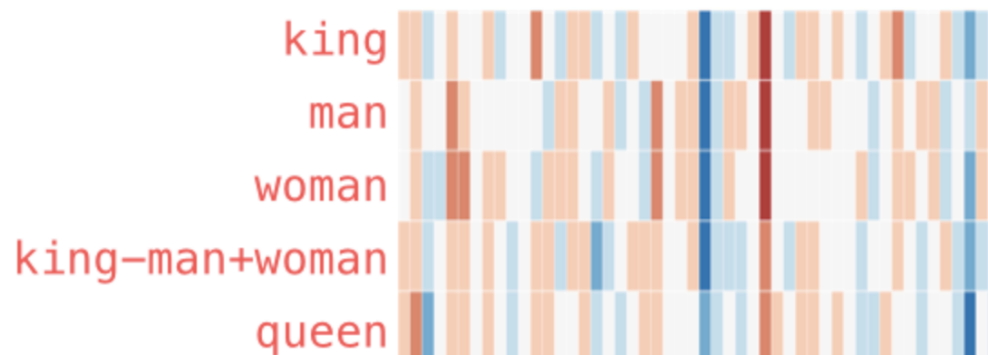
[The Illustrated Word2vec](#)

Operations using word embeddings

```
model.most_similar(positive=["king", "woman"], negative=["man"])
```

```
[('queen', 0.8523603677749634),  
 ('throne', 0.7664333581924438),  
 ('prince', 0.7592144012451172),  
 ('daughter', 0.7473883032798767),  
 ('elizabeth', 0.7460219860076904),  
 ('princess', 0.7424570322036743),  
 ('kingdom', 0.7337411642074585),  
 ('monarch', 0.721449077129364),  
 ('eldest', 0.7184862494468689),  
 ('widow', 0.7099430561065674)]
```

king - man + woman $\approx$ queen



Problem with static embeddings (word2vec)

- They are static! The embedding for a word doesn't reflect how its meaning changes in context.

- The chicken didn't cross the road because  it was too tired

- What is the meaning represented in the static embedding for "it"?

Contextual Embeddings

- Intuition: a representation of meaning of a word should be different in different contexts!
- **Contextual Embedding:** each word has a different vector that expresses different meanings depending on the surrounding words

Contextual Embeddings

- The chicken didn't cross the road because it

- What should be the properties of "it"?

- The chicken didn't cross the road because it was too **tired**

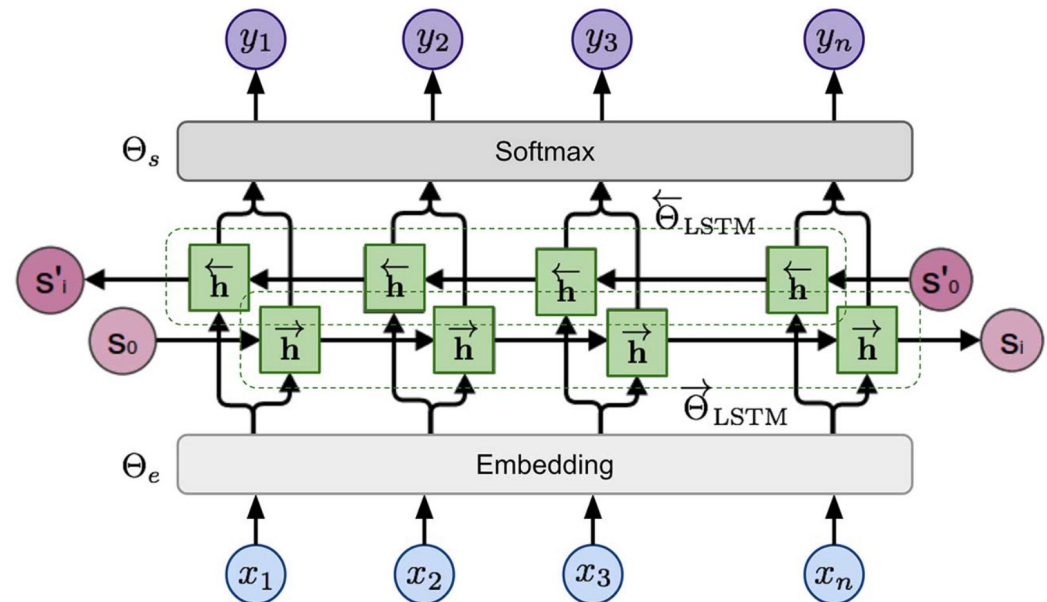
- The chicken didn't cross the road because it was too **wide**

- At this point in the sentence, it's probably referring to either the chicken or the street

- But the static meaning of "it" can't represent that, it just means "I am a non-human pronoun"

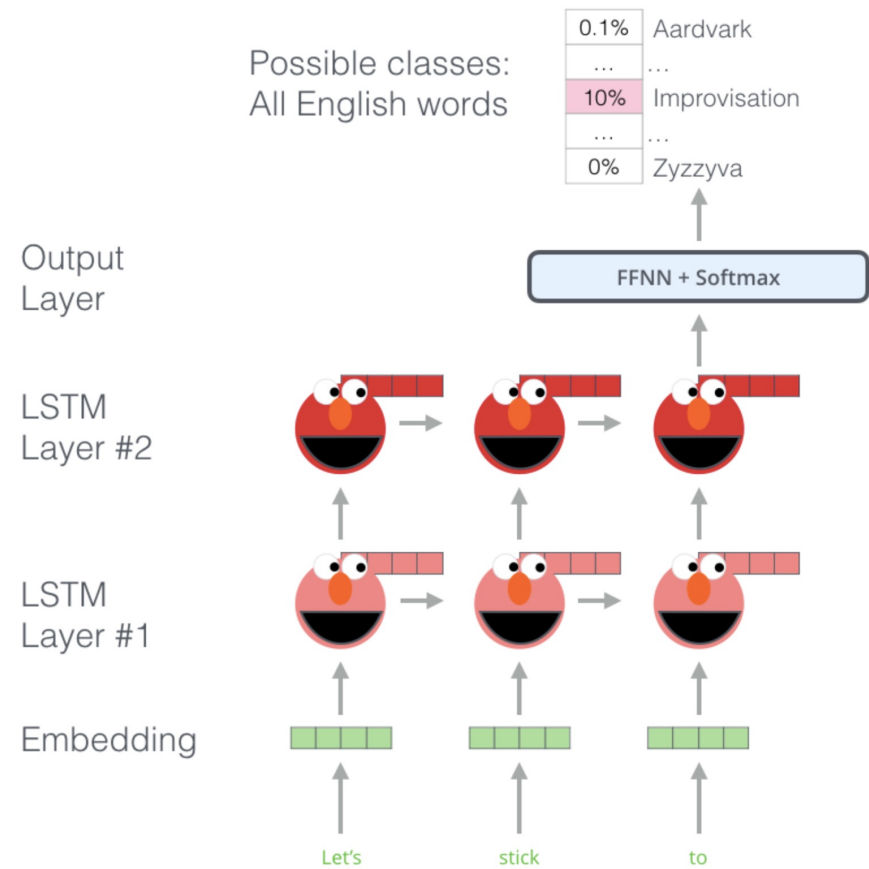
ELMo: Context Matters

- ELMo generates *contextualized* embeddings for a word
- Give words different embeddings based on the meaning they carry in the context of the sentence
- LSTM based architecture



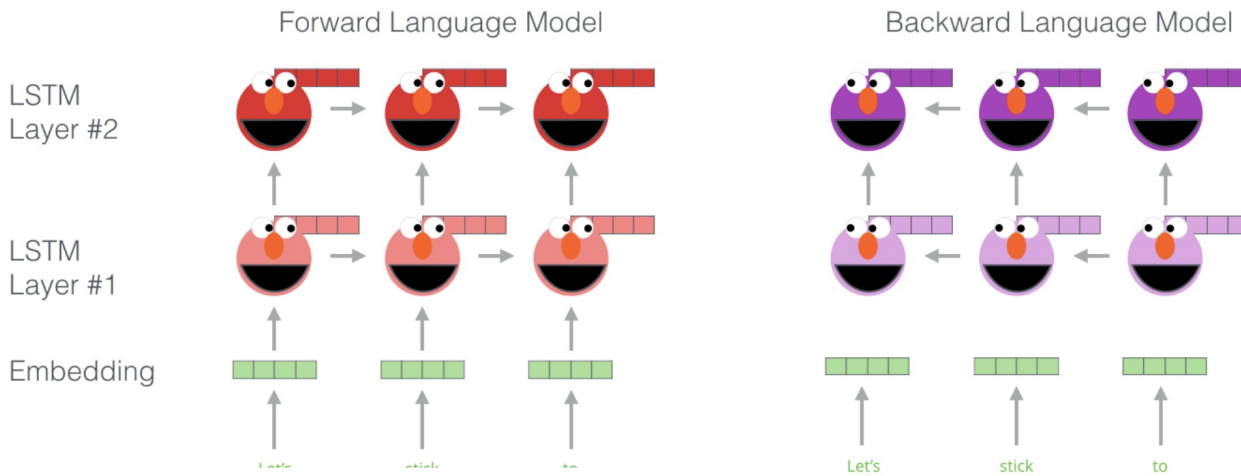
ELMo Training

- Predict next word in a sequence of words
- Self-supervised learning, without any need for labels



Embedding generation in ELMo: Step 1

Embedding of “stick” in “Let’s stick to” - Step #1



Training objective: Jointly maximizes the log likelihood of the forward and backward directions

$$\sum_{k=1}^N (\log p(t_k \mid t_1, \dots, t_{k-1}; \Theta_x, \vec{\Theta}_{LSTM}, \Theta_s) + \log p(t_k \mid t_{k+1}, \dots, t_N; \Theta_x, \overleftarrow{\Theta}_{LSTM}, \Theta_s))$$

$$p(t_1, t_2, \dots, t_N) = \prod_{k=1}^N p(t_k \mid t_1, t_2, \dots, t_{k-1})$$

$$p(t_1, t_2, \dots, t_N) = \prod_{k=1}^N p(t_k \mid t_{k+1}, t_{k+2}, \dots, t_N)$$

Embedding generation in ELMo: Step 2

- Contextualized embedding through grouping together the hidden states (and initial embedding)

Embedding of “stick” in “Let’s stick to” - Step #2

1- Concatenate hidden layers



2- Multiply each vector by a weight based on the task

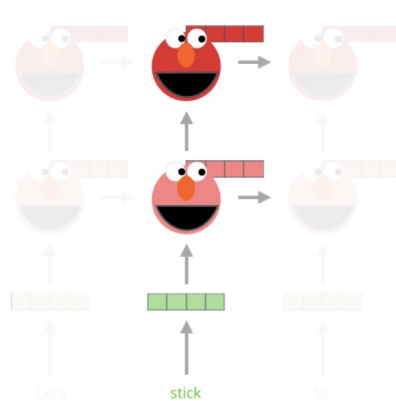


3- Sum the (now weighted) vectors

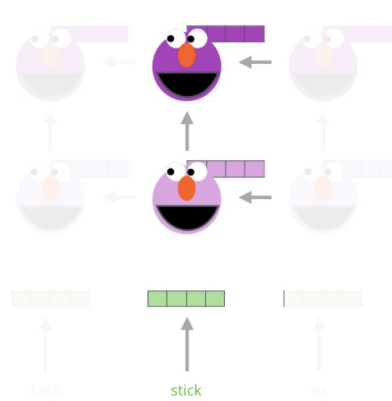


ELMo embedding of “stick” for this task in this context

Forward Language Model



Backward Language Model



each token t_k , a L -layer biLM computes a set of $2L + 1$ representations

$$R_k = \{\mathbf{x}_k^{LM}, \vec{\mathbf{h}}_{k,j}^{LM}, \overleftarrow{\mathbf{h}}_{k,j}^{LM} \mid j = 1, \dots, L\}$$

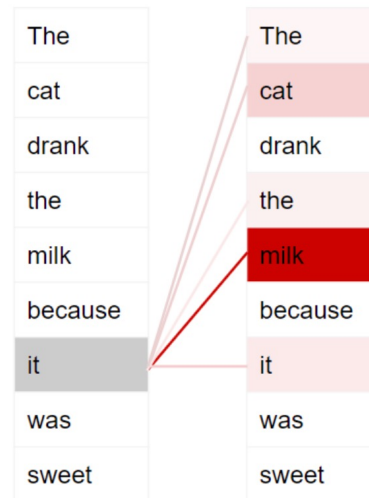
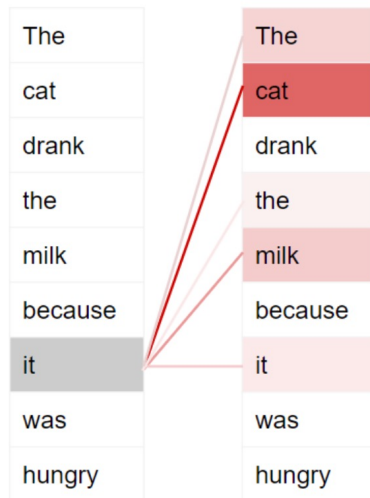
$$= \{\mathbf{h}_{k,j}^{LM} \mid j = 0, \dots, L\},$$

Task specific ELMo embedding

$$\mathbf{ELMo}_k^{task} = E(R_k; \Theta^{task}) = \gamma^{task} \sum_{j=0}^L s_j^{task} \mathbf{h}_{k,j}^{LM}$$

Motivating Attention

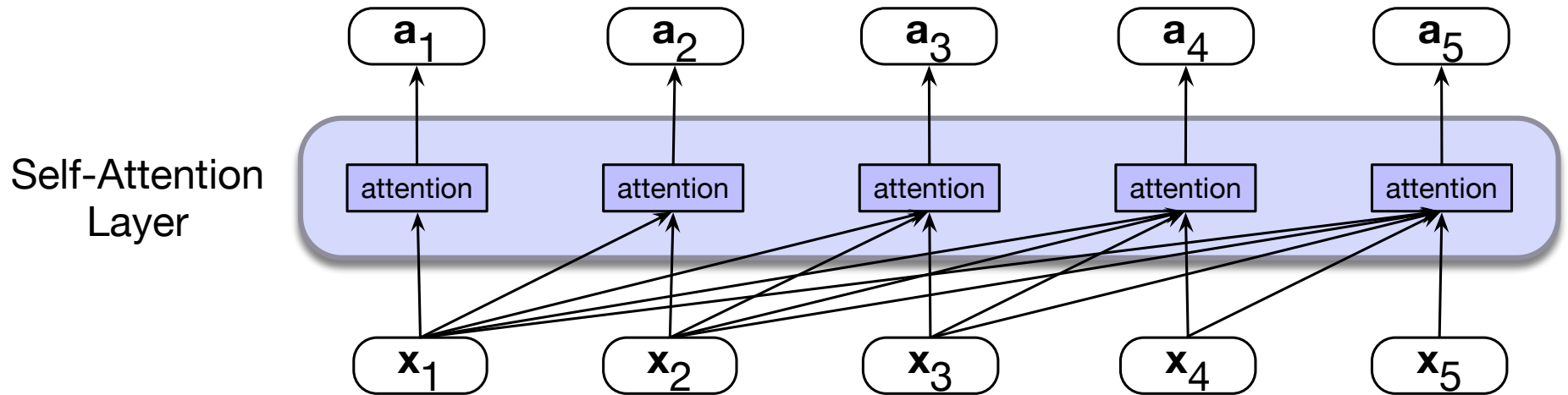
- Capture long-range dependence in context
- Example:
 1. The *cat* drank the milk because **it** was hungry.
 2. The cat drank the *milk* because **it** was sweet.



Attention definition

- A mechanism for helping compute the embedding for a token by selectively attending to and integrating information from surrounding tokens (at the previous layer).
- More formally: a method for doing a weighted sum of vectors.

Left-to-right Attention



Simplified version of attention: a sum of prior words weighted by their similarity with the current word

Given a sequence of token embeddings:

$$\mathbf{x}_1 \quad \mathbf{x}_2 \quad \mathbf{x}_3 \quad \mathbf{x}_4 \quad \mathbf{x}_5 \quad \mathbf{x}_6 \quad \mathbf{x}_7 \quad \mathbf{x}_i$$

Produce: $\mathbf{a}_i$ = a weighted sum of $\mathbf{x}_1$ through $\mathbf{x}_7$ (and $\mathbf{x}_i$)

Weighted by their similarity to $\mathbf{x}_i$

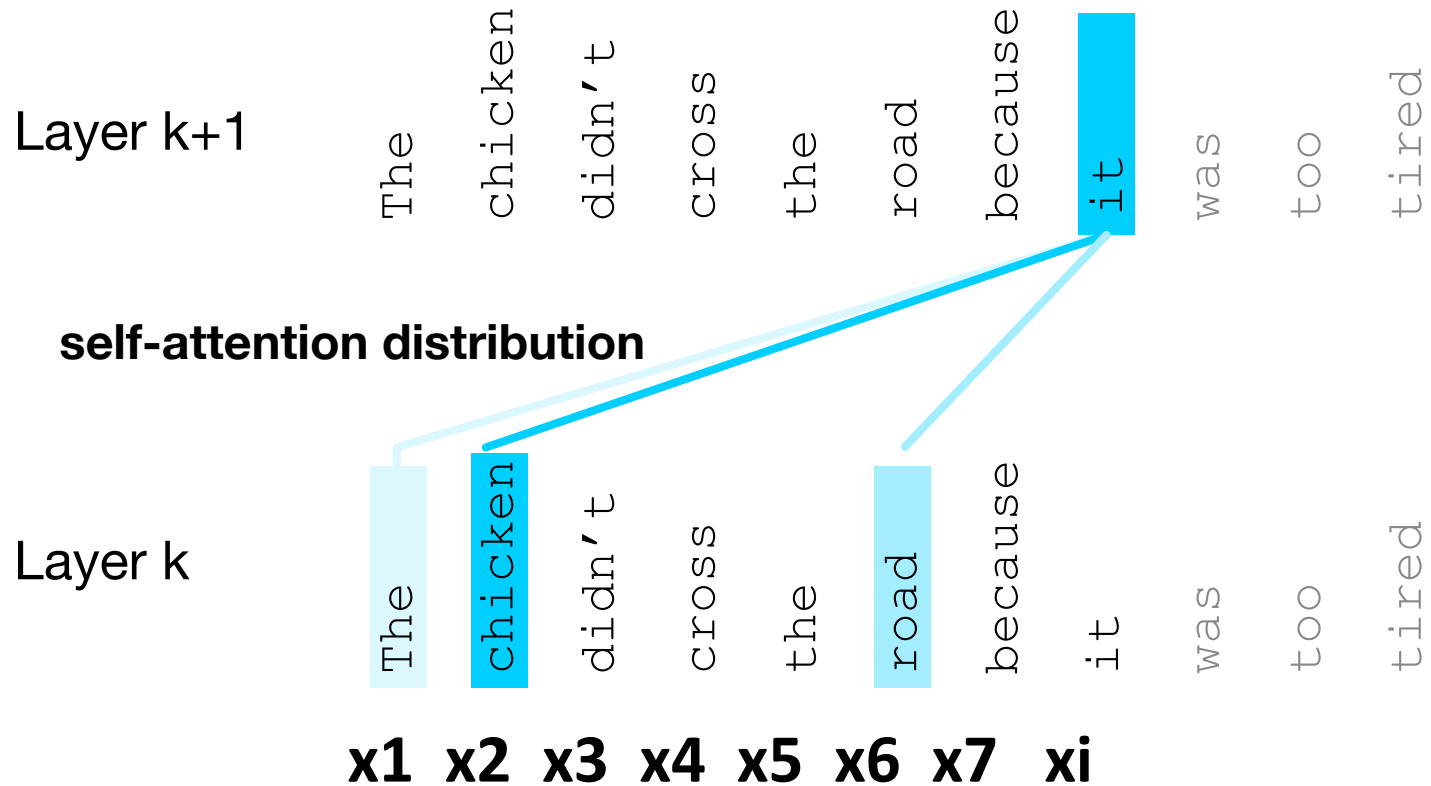
$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i$$

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j$$

Intuition of attention:

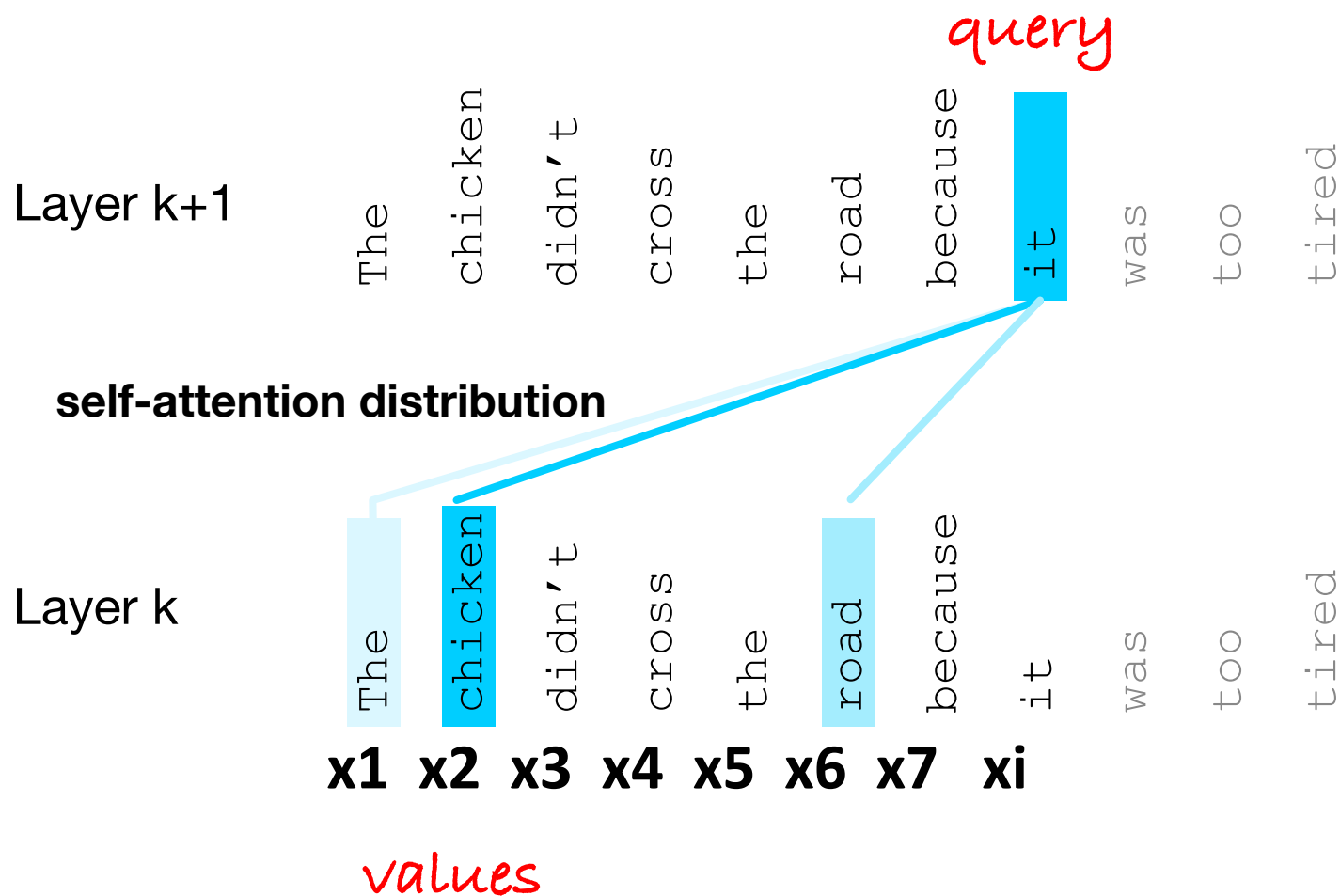
columns corresponding to input tokens



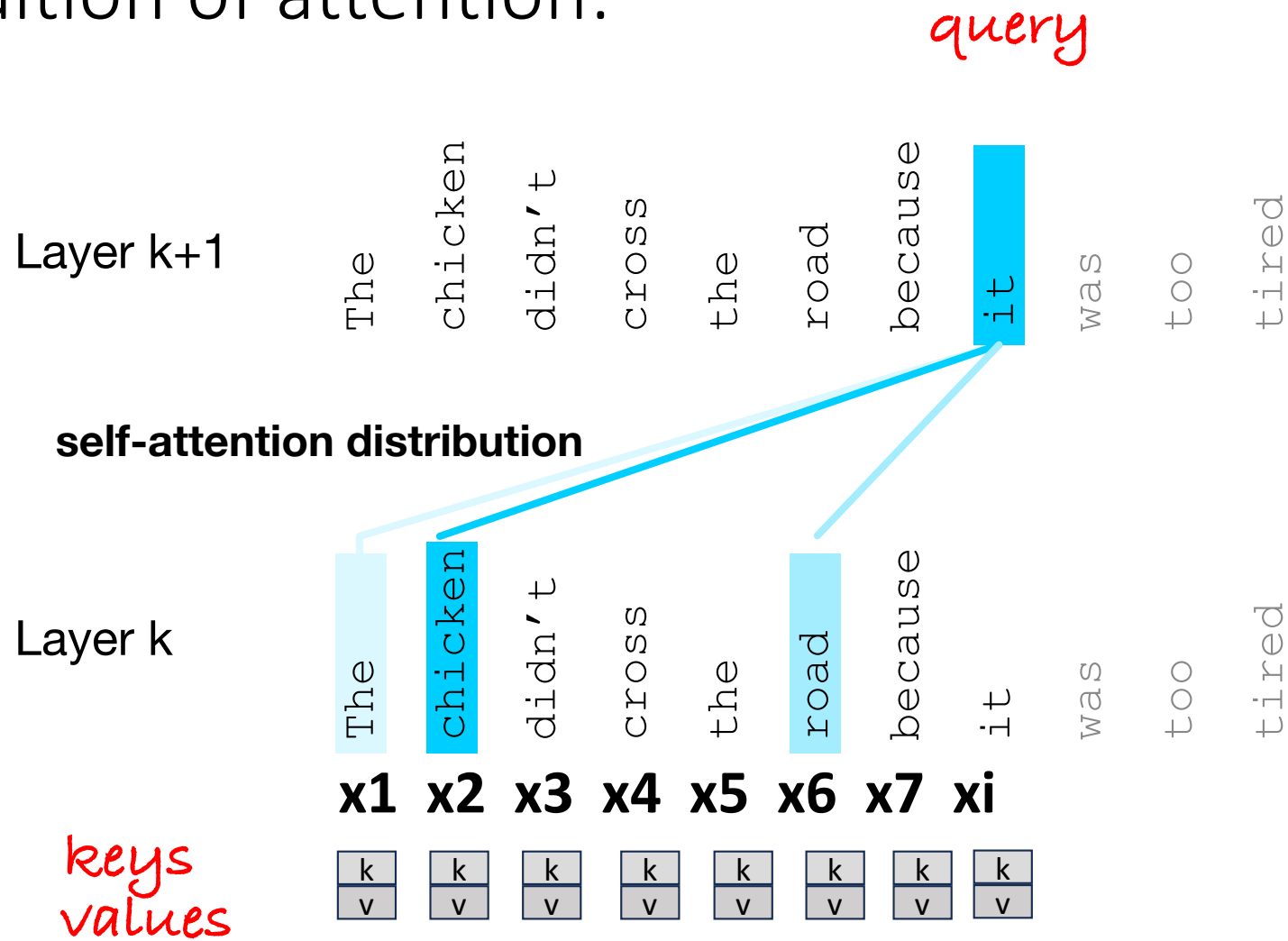
An Actual Attention Head: slightly more complicated

- High-level idea: instead of using vectors (like x_i and x_4) directly, we'll represent 3 separate roles each vector x_i plays:
- **query**: *As the current element* being compared to the preceding inputs.
- **key**: *as a preceding input* that is being compared to the current element to determine a similarity
- **value**: a value of a preceding element that gets weighted and summed

Attention intuition



Intuition of attention:



An Actual Attention Head: slightly more complicated

- We'll use matrices to project each vector $\mathbf{x}_i$ into a representation of its role as query, key, value:
- **query:** $\mathbf{W}^Q$
- **key:** $\mathbf{W}^K$
- **value:** $\mathbf{W}^V$

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

An Actual Attention Head: slightly more complicated

- Given these 3 representations of $\mathbf{x}_i$
$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$
- To compute similarity of current element $\mathbf{x}_i$ with some prior element $\mathbf{x}_j$
- We'll use dot product between $\mathbf{q}_i$ and $\mathbf{k}_j$.
- And instead of summing up $\mathbf{x}_j$, we'll sum up $\mathbf{v}_j$

Final equations for one attention head

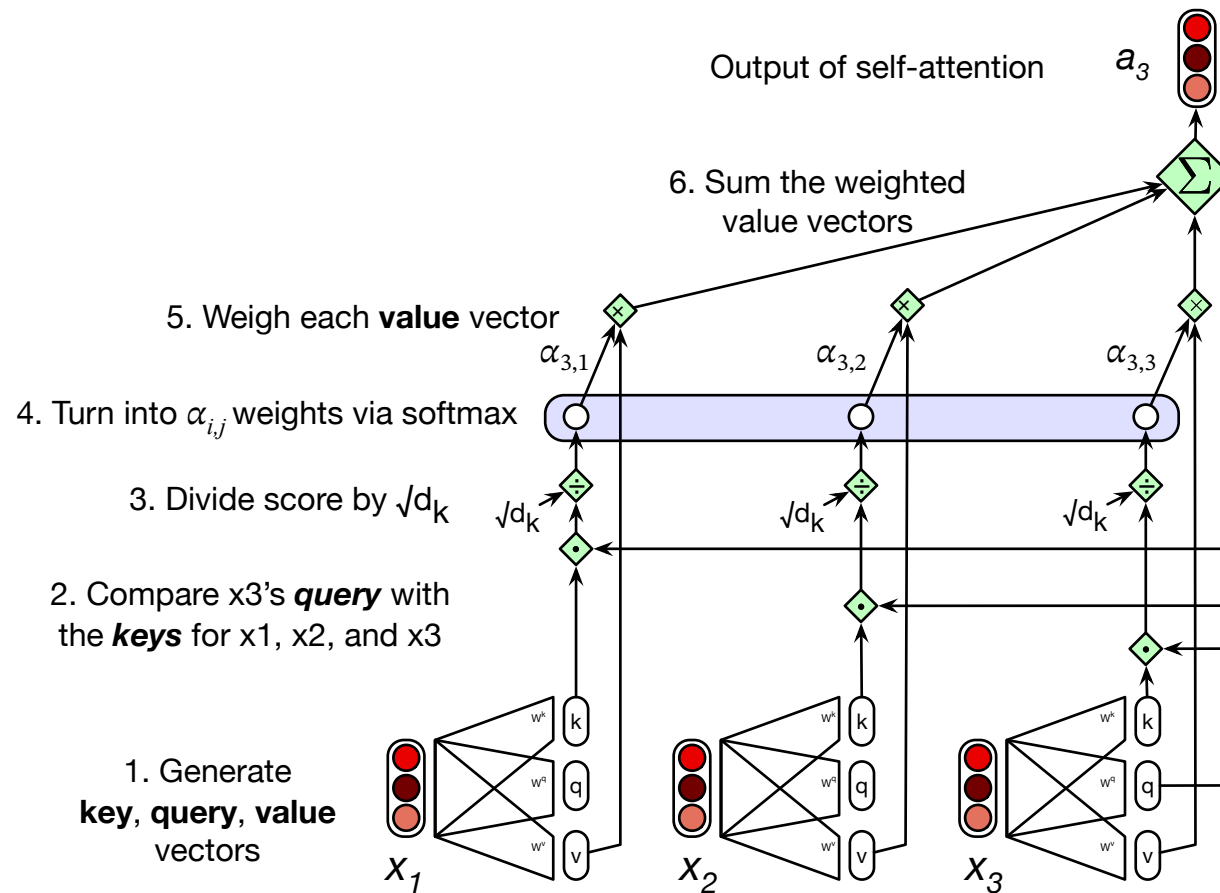
$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_j = \mathbf{x}_j \mathbf{W}^K; \quad \mathbf{v}_j = \mathbf{x}_j \mathbf{W}^V$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i$$

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j$$

Calculating the value of a_3



Actual Attention: slightly more complicated

- Instead of one attention head, we'll have lots of them!
- Intuition: each head might be attending to the context for different purposes
 - Different linguistic relationships or patterns in the context

$$\mathbf{q}_i^c = \mathbf{x}_i \mathbf{W}^{\mathbf{Q}^c}; \quad \mathbf{k}_j^c = \mathbf{x}_j \mathbf{W}^{\mathbf{K}^c}; \quad \mathbf{v}_j^c = \mathbf{x}_j \mathbf{W}^{\mathbf{V}^c}; \quad \forall c \quad 1 \leq c \leq h$$

$$\text{score}^c(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i^c \cdot \mathbf{k}_j^c}{\sqrt{d_k}}$$

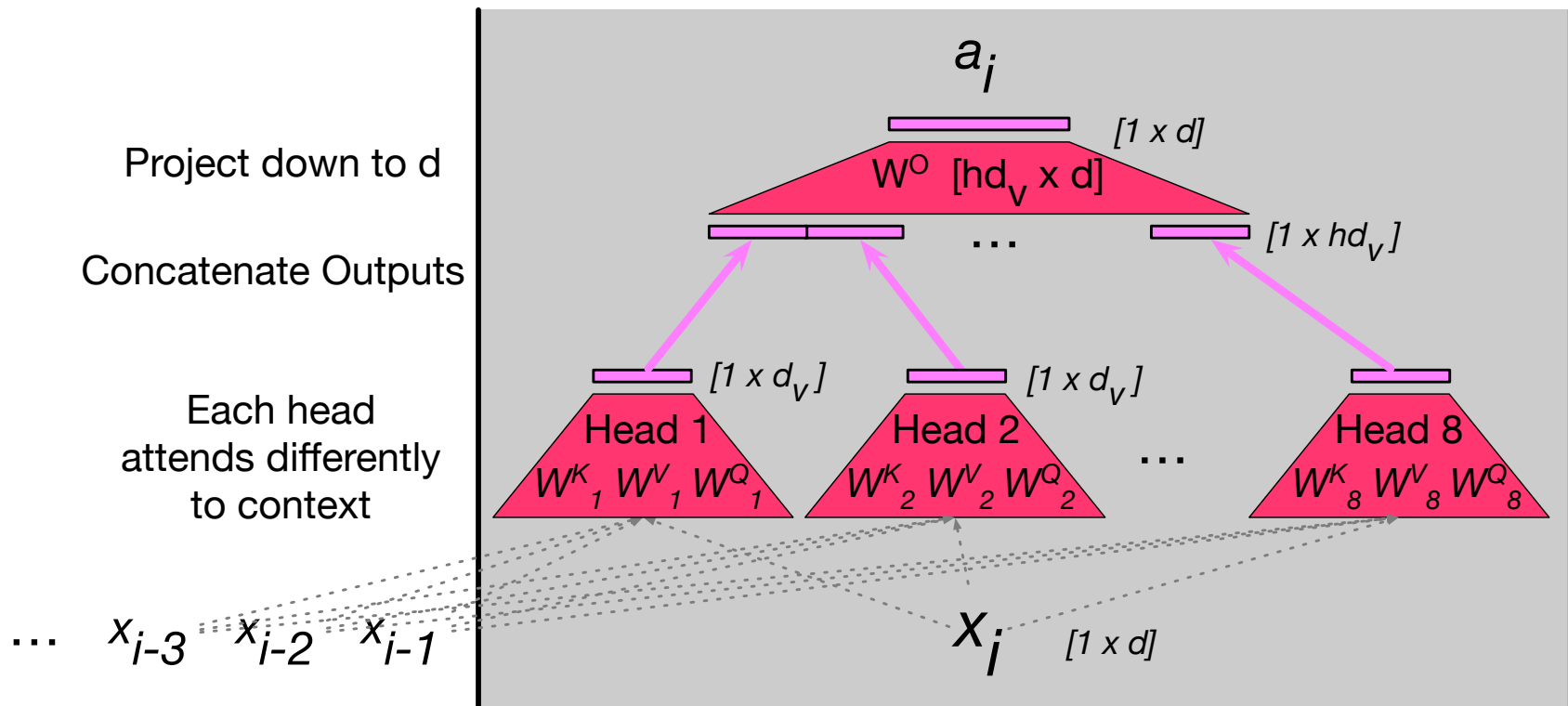
$$\alpha_{ij}^c = \text{softmax}(\text{score}^c(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i$$

$$\text{head}_i^c = \sum_{j \leq i} \alpha_{ij}^c \mathbf{v}_j^c$$

$$\mathbf{a}_i = (\text{head}^1 \oplus \text{head}^2 \dots \oplus \text{head}^h) \mathbf{W}^O$$

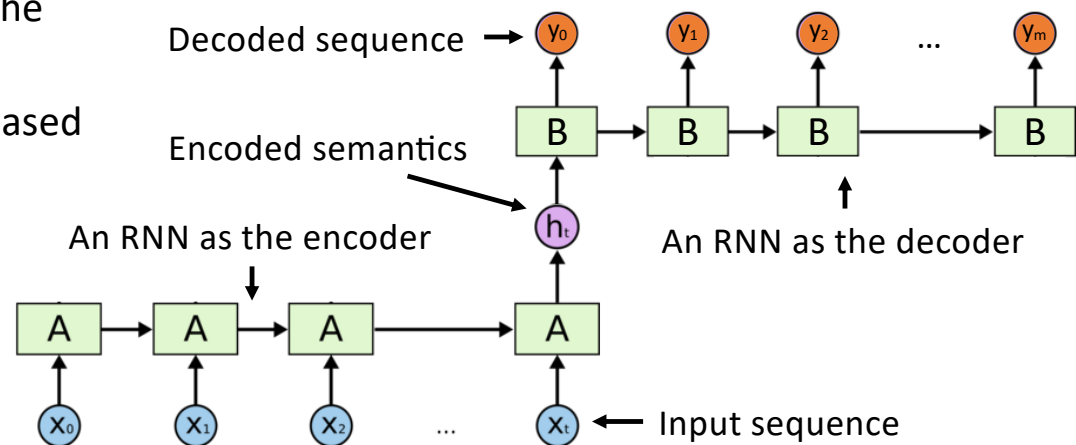
$$\text{MultiHeadAttention}(\mathbf{x}_i, [\mathbf{x}_1, \dots, \mathbf{x}_N]) = \mathbf{a}_i$$

Multi-head attention

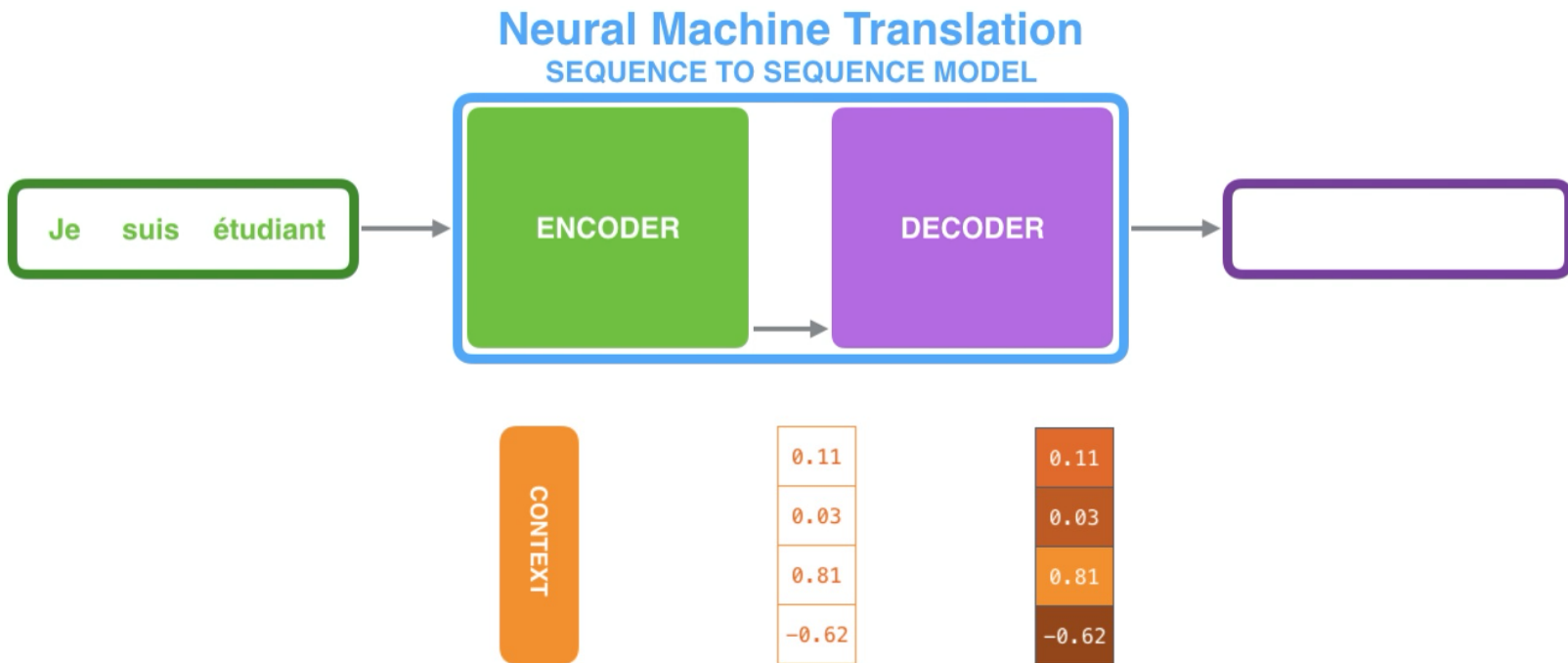


Sequence-to-Sequence (Seq2Seq) model

- Developed by Google in 2018 for use in machine translation.
- Seq2seq turns one sequence into another sequence. It does so by use of a [recurrent neural network](#) (RNN) or more often [LSTM](#) or [GRU](#) to avoid the problem of [vanishing gradient](#).
- The primary components are one Encoder and one Decoder network. The encoder turns each item into a corresponding hidden vector containing the item and its context. The decoder reverses the process, turning the vector into an output item, using the previous output as the input context.
- **Encoder RNN:** extract and compress the semantics from the input sequence
- **Decoder RNN:** generate a sequence based on the input semantics
- Apply to tasks such as machine translation
 - Similar underlying semantics
 - E.g., “I love you.” to “Je t’aime.”



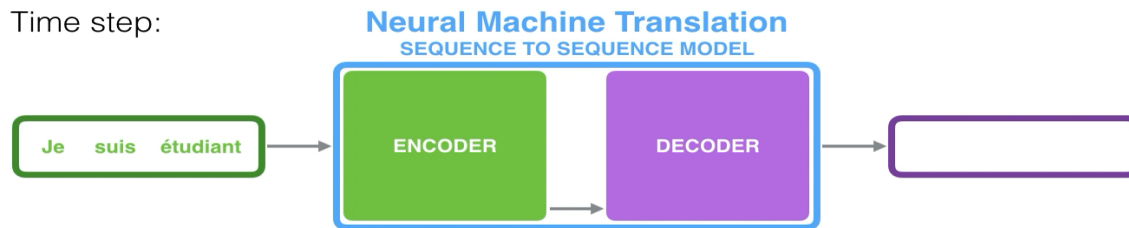
Context in Seq2Seq model



[Visualizing A Neural Machine Translation Model \(Mechanics of Seq2seq Models With Attention\)](#)

Hidden states as context

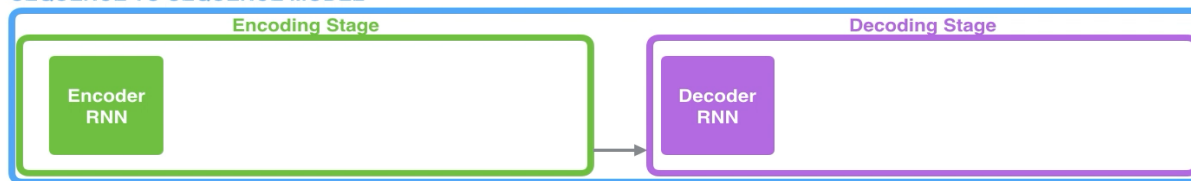
Time step:



The last **hidden state** of the **encoder** forms the **context** we pass along to the **decoder**.

The **decoder** also maintains a **hidden state** that it passes from one time step to the next.

Neural Machine Translation SEQUENCE TO SEQUENCE MODEL



Je suis étudiant

[Visualizing A Neural Machine Translation Model \(Mechanics of Seq2seq Models With Attention\)](#)

Attention in Seq2Seq model

- The **context** vector turned out to be a bottleneck for Seq2Seq models
- “Attention” mechanism was first proposed in [Bahdanau et al., 2014](#) and [Luong et al., 2015](#)
- Attention allows a Seq2Seq model to focus on the relevant parts of the input sequence when decoding
- Attention model differs from a classic Seq2Seq model:
 1. The **encoder** passes *all* the **hidden states** to the **decoder**
 2. The **decoder** gives different scores to different hidden states from the **encoder**

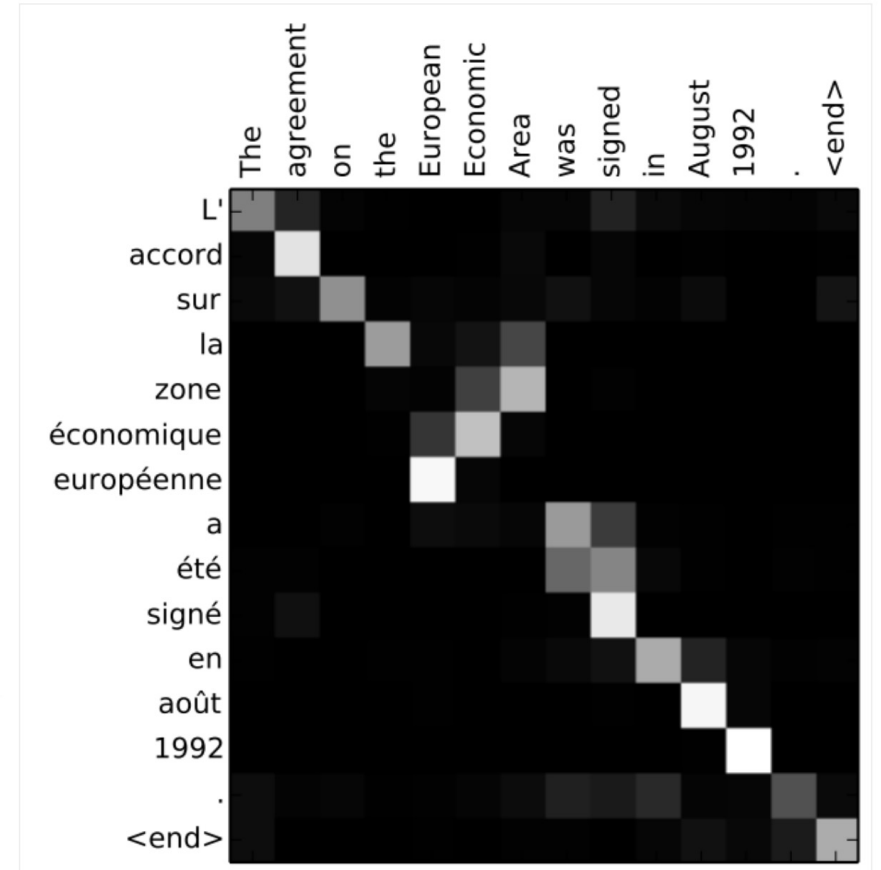
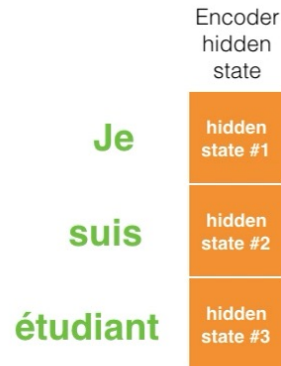
Attention Mechanism in Seq2Seq model

Attention at time step 4



[Visualizing A Neural Machine Translation Model \(Mechanics of Seq2seq Models With Attention\)](#)

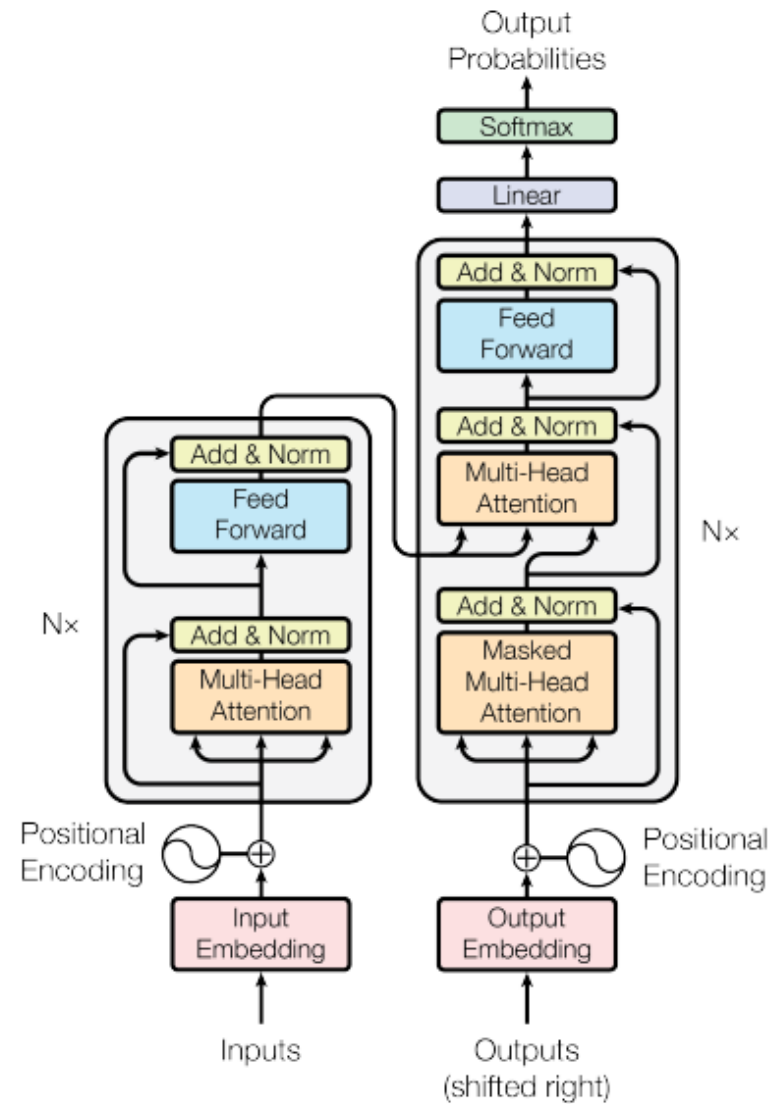
Attention example



You can see how the model paid attention correctly when outputting "European Economic Area". In French, the order of these words is reversed ("européenne économique zone") as compared to English. Every other word in the sentence is in similar order.

Transformer

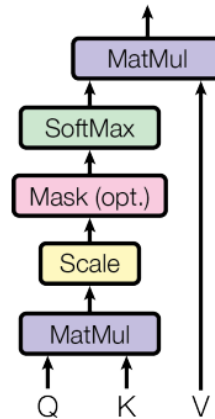
- [Attention is all you need](#)
- Encoder—Decoder architecture
- The Illustrated Transformer



Multi-Head Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Scaled Dot-Product Attention



Multi-Head Attention

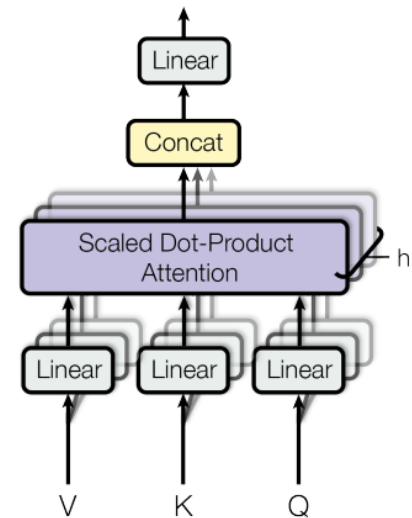
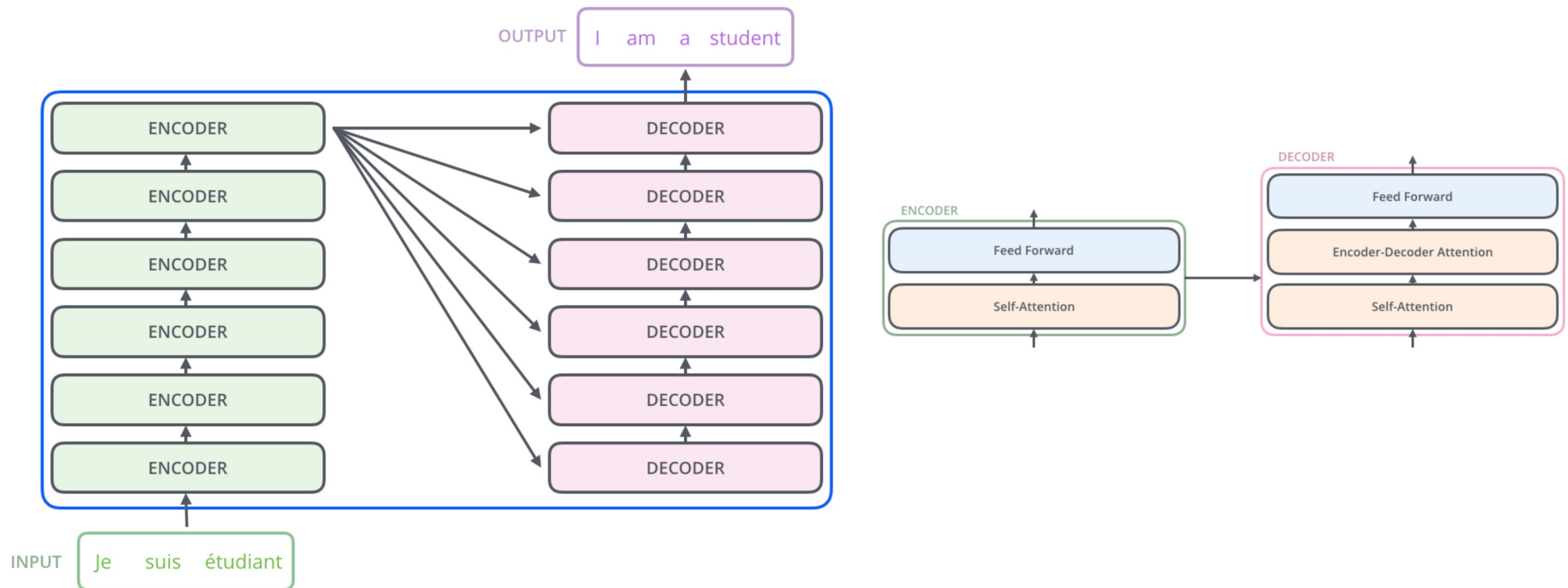


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

Attention in Transformer models

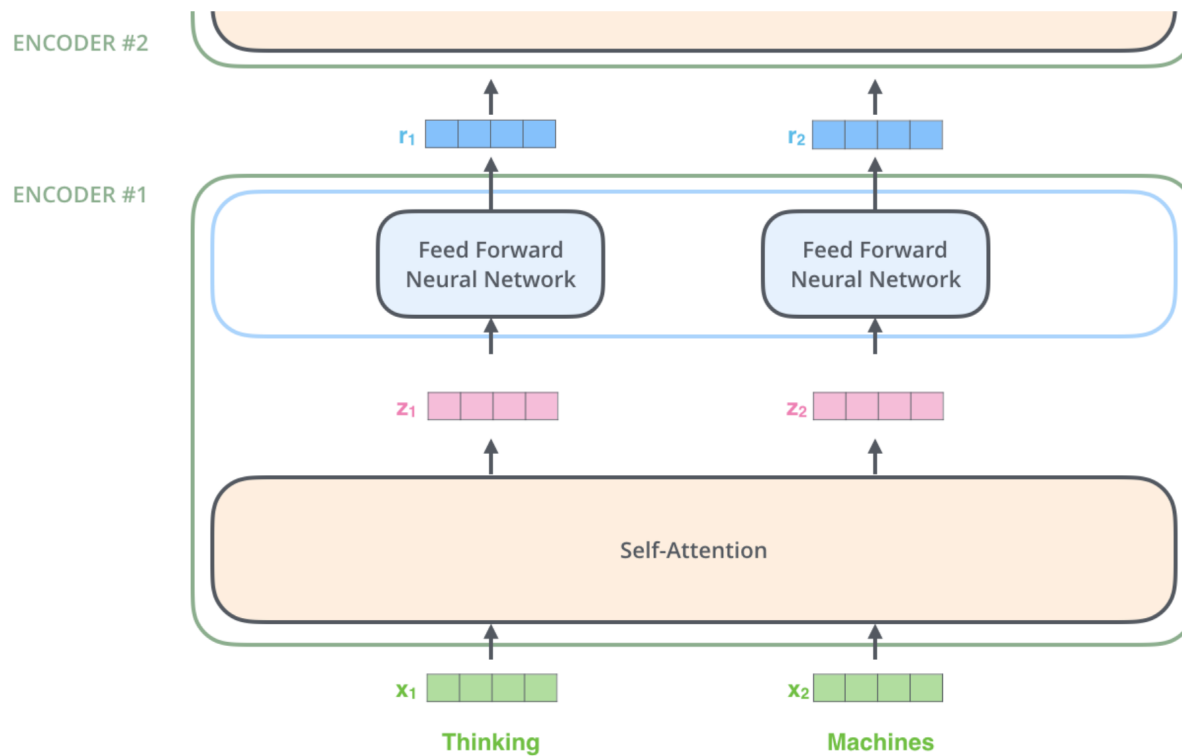
- The Transformer uses multi-head attention in three different ways:
- In "encoder-decoder attention" layers, the queries come from the previous decoder layer, and the memory keys and values come from the output of the encoder. This allows every position in the decoder to attend over all positions in the input sequence. This mimics the typical encoder-decoder attention mechanisms in sequence-to-sequence models.
- The encoder contains self-attention layers. In a self-attention layer all of the keys, values and queries come from the same place, in this case, the output of the previous layer in the encoder. Each position in the encoder can attend to all positions in the previous layer of the encoder.
- Similarly, self-attention layers in the decoder allow each position in the decoder to attend to all positions in the decoder up to and including that position. We need to prevent leftward information flow in the decoder to preserve the auto-regressive property. We implement this inside of scaled dot-product attention by masking out (setting to $-\infty$) all values in the input of the softmax which correspond to illegal connections.

Transformer – a parallelizable encoder-decoder architecture



[The Illustrated Transformer – Part2](#)

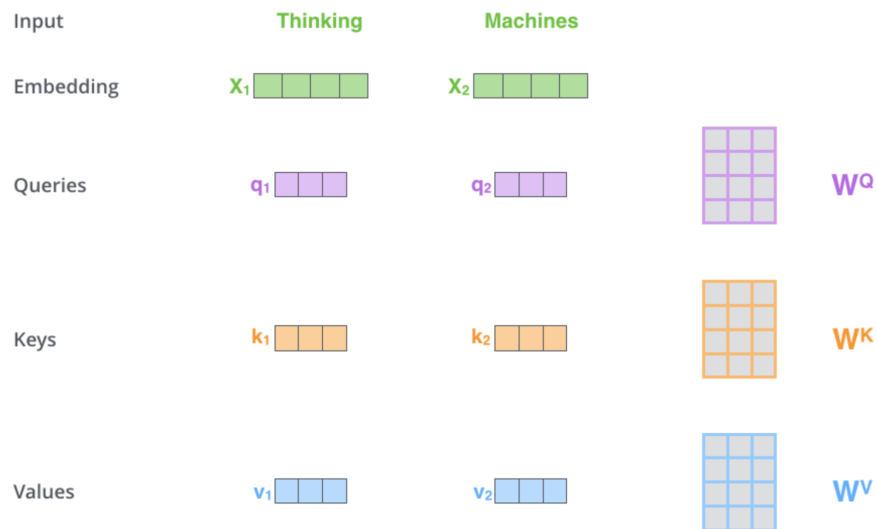
Flow of Tensors in Transformers



The word at each position passes through a self-attention process. Then, they each pass through a feed-forward neural network -- the exact same network with each vector flowing through it separately.

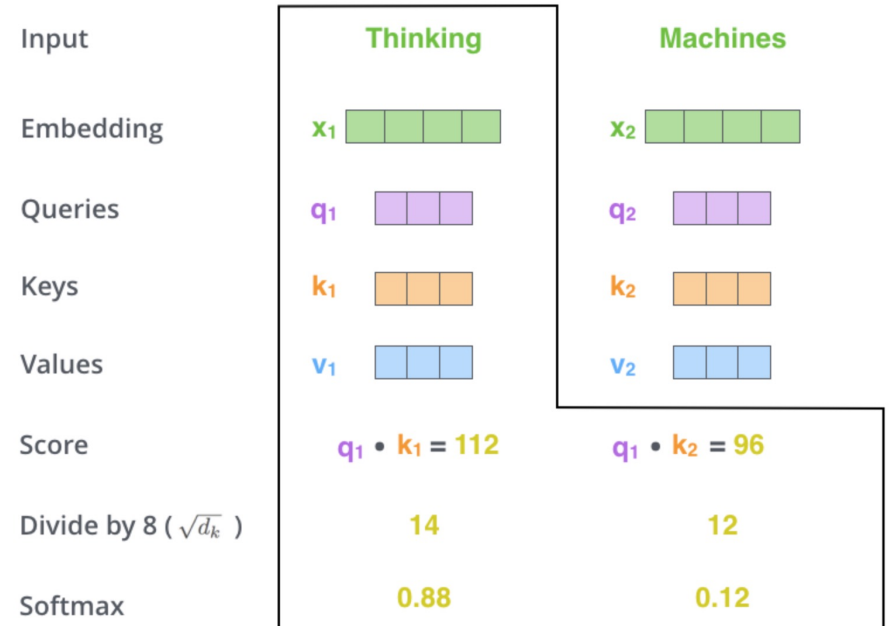
Self-Attention Steps

1. create three vectors (query, key, value) from each of the encoder's input vectors



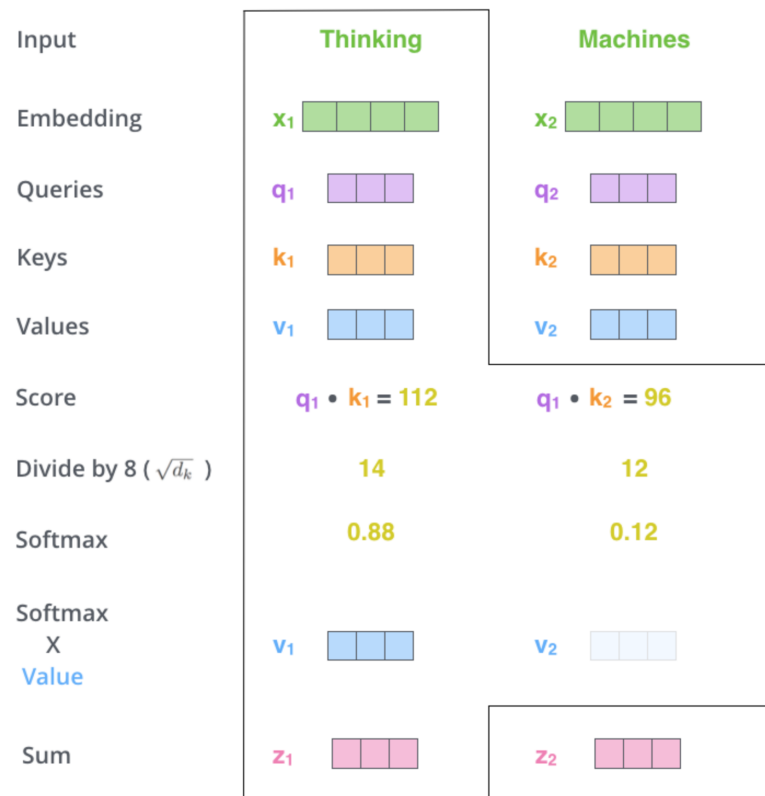
Multiplying x1 by the WQ weight matrix produces q1, the "query" vector associated with that word. We end up creating a "query", a "key", and a "value" projection of each word in the input sentence.

2. create a score for each input vector and pass normalized score through a softmax



Self-Attention Steps

3. multiply each value vector by the softmax score and sum the weighted value vectors



Matrix calculation of self-attention

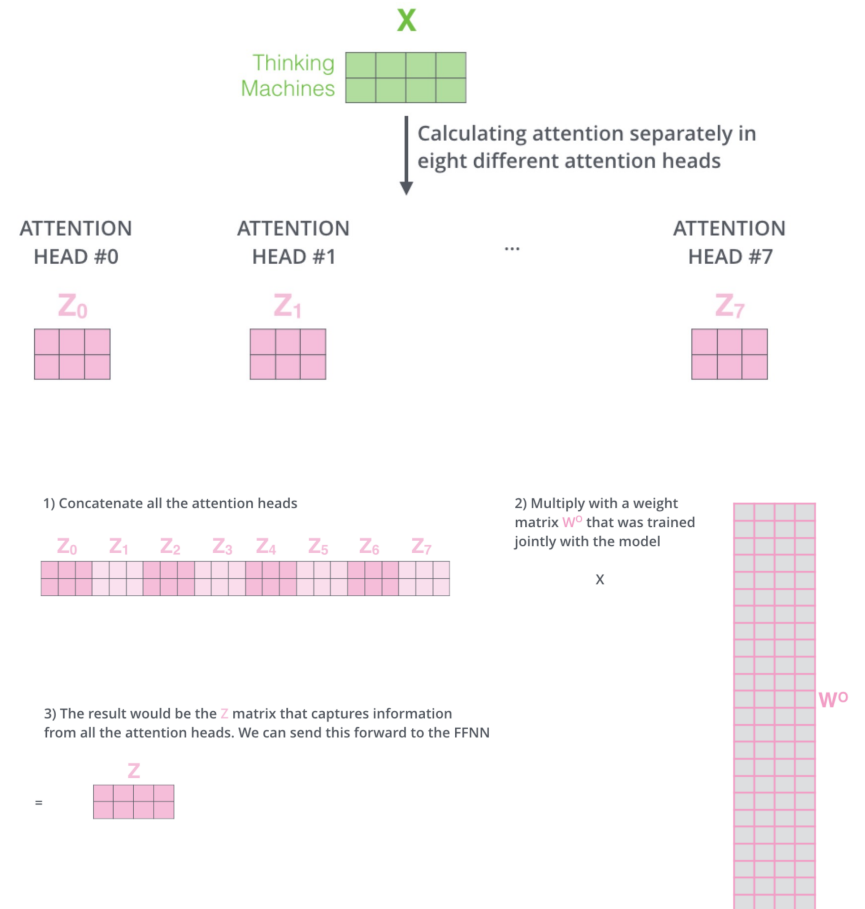
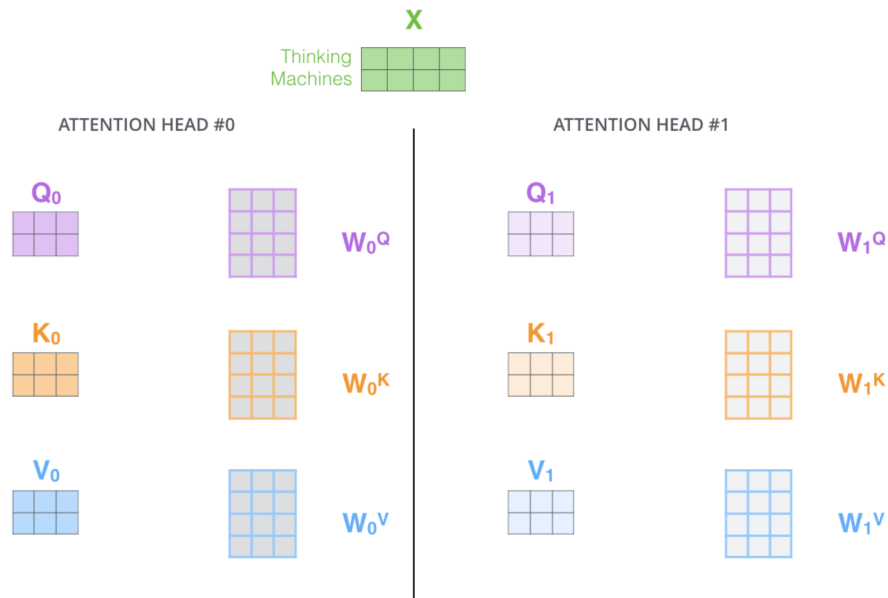
$$\begin{matrix} \text{X} \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{W}^{\text{Q}} \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline \end{array} \end{matrix} = \begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline \end{array} \end{matrix}$$

$$\begin{matrix} \text{X} \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{W}^{\text{K}} \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline \end{array} \end{matrix} = \begin{matrix} \text{K} \\ \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline \end{array} \end{matrix}$$

$$\begin{matrix} \text{X} \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{W}^{\text{V}} \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline \end{array} \end{matrix} = \begin{matrix} \text{V} \\ \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline \end{array} \end{matrix}$$

$$\begin{aligned} & \text{softmax} \left(\frac{\begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{K}^{\text{T}} \\ \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline \end{array} \end{matrix}}{\sqrt{d_k}} \right) \begin{matrix} \text{V} \\ \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline \end{array} \end{matrix} \\ & = \begin{matrix} \text{Z} \\ \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline \end{array} \end{matrix} \end{aligned}$$

Multi-head Attention



Self attention summary

1) This is our input sentence*

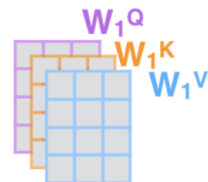
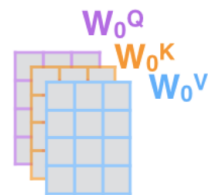
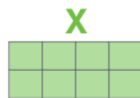
2) We embed each word*

3) Split into 8 heads.
We multiply X or R with weight matrices

4) Calculate attention using the resulting $Q/K/V$ matrices

5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

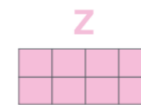
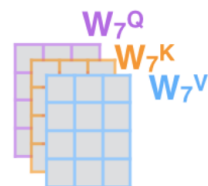
Thinking
Machines



...

...

...



* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

Transformers

Parallelizing Attention Computation

Parallelizing computation using $\mathbf{X}$

For attention/transformer block we've been computing a **single** output at a **single** time step i in a **single** residual stream.

But we can pack the N tokens of the input sequence into a single matrix $\mathbf{X}$ of size $[N \times d]$.

Each row of $\mathbf{X}$ is the embedding of one token of the input.

$\mathbf{X}$ can have 1K-128K rows, each of the dimensionality of the embedding d (the **model dimension**)

$$\mathbf{Q} = \mathbf{XW}^{\mathbf{Q}}; \quad \mathbf{K} = \mathbf{XW}^{\mathbf{K}}; \quad \mathbf{V} = \mathbf{XW}^{\mathbf{V}}$$


$$QK^T$$

Now can do a single matrix multiply to combine Q and K^T

N	q1•k1	q1•k2	q1•k3	q1•k4
	q2•k1	q2•k2	q2•k3	q2•k4
	q3•k1	q3•k2	q3•k3	q3•k4
	q4•k1	q4•k2	q4•k3	q4•k4
N				

Parallelizing attention

- Scale the scores, take the softmax, and then multiply the result by V resulting in a matrix of shape $N \times d$
 - An attention vector for each input token

$$\mathbf{A} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}} \right) \right) \mathbf{V}$$

Masking out the future

$$\mathbf{A} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} \right) \right) \mathbf{V}$$

- What is this mask function?
 $\mathbf{Q}\mathbf{K}^\top$ has a score for each query dot every key, *including those that follow the query.*
- Guessing the next word is pretty simple if you already know it!

Masking out the future

$$\mathbf{A} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} \right) \right) \mathbf{V}$$

Add $-\infty$ to cells in upper triangle
The softmax will turn it to 0

N

q1•k1	$-\infty$	$-\infty$	$-\infty$
q2•k1	q2•k2	$-\infty$	$-\infty$
q3•k1	q3•k2	q3•k3	$-\infty$
q4•k1	q4•k2	q4•k3	q4•k4

N

Another point: Attention is quadratic in length

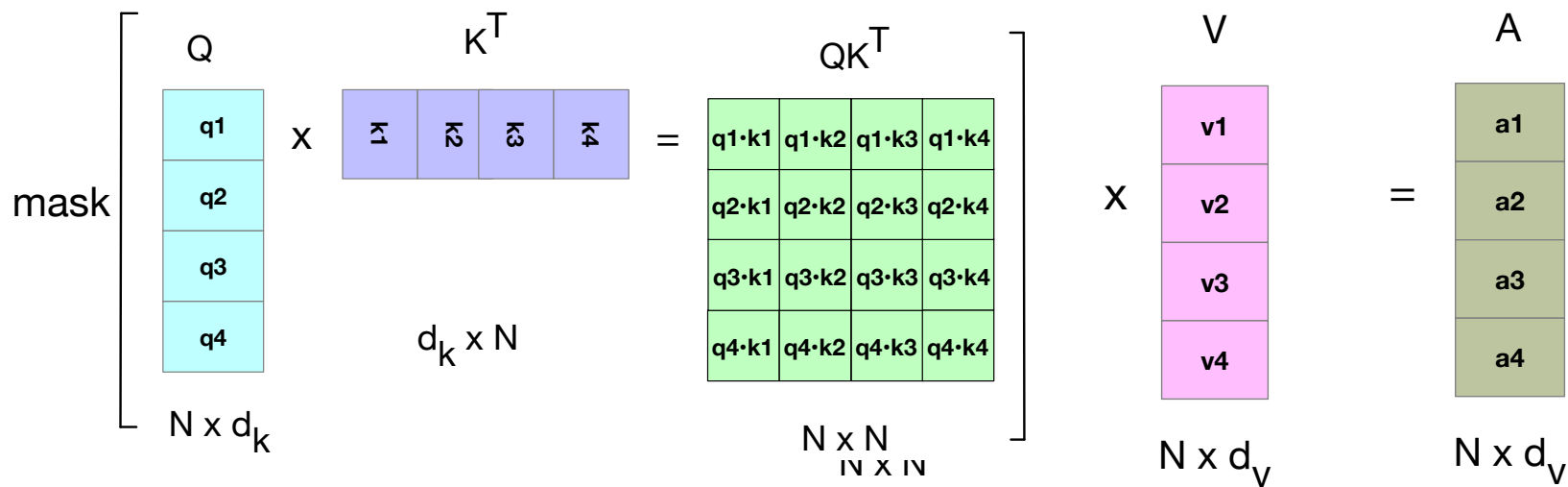
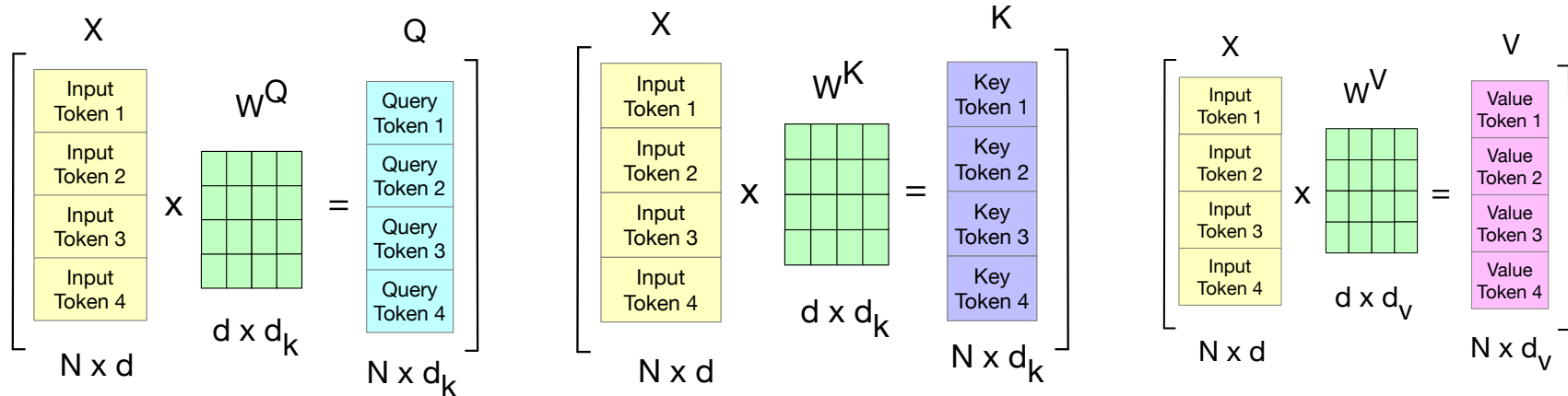
$$\mathbf{A} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} \right) \right) \mathbf{V}$$

N

q1•k1	−∞	−∞	−∞
q2•k1	q2•k2	−∞	−∞
q3•k1	q3•k2	q3•k3	−∞
q4•k1	q4•k2	q4•k3	q4•k4

N

Attention again



Parallelizing Multi-head Attention

$$\mathbf{Q}^i = \mathbf{XW}^{\mathbf{Q}^i}; \quad \mathbf{K}^i = \mathbf{XW}^{\mathbf{K}^i}; \quad \mathbf{V}^i = \mathbf{XW}^{\mathbf{V}^i}$$

$$\mathbf{head}_i = \text{SelfAttention}(\mathbf{Q}^i, \mathbf{K}^i, \mathbf{V}^i) = \text{softmax}\left(\frac{\mathbf{Q}^i \mathbf{K}^{i\top}}{\sqrt{d_k}}\right) \mathbf{V}^i$$

$$\text{MultiHeadAttention}(\mathbf{X}) = (\mathbf{head}_1 \oplus \mathbf{head}_2 \dots \oplus \mathbf{head}_h) \mathbf{W}^0$$

Parallelizing Multi-head Attention

$$\mathbf{O} = \text{LayerNorm}(\mathbf{X} + \text{MultiHeadAttention}(\mathbf{X}))$$

$$\mathbf{H} = \text{LayerNorm}(\mathbf{O} + \text{FFN}(\mathbf{O}))$$

or

$$\mathbf{T}^1 = \text{MultiHeadAttention}(\mathbf{X})$$

$$\mathbf{T}^2 = \mathbf{X} + \mathbf{T}^1$$

$$\mathbf{T}^3 = \text{LayerNorm}(\mathbf{T}^2)$$

$$\mathbf{T}^4 = \text{FFN}(\mathbf{T}^3)$$

$$\mathbf{T}^5 = \mathbf{T}^4 + \mathbf{T}^3$$

$$\mathbf{H} = \text{LayerNorm}(\mathbf{T}^5)$$

Transformers

Input and output: Position embeddings and the Language Model Head



Token and Position Embeddings

The matrix X (of shape $[N \times d]$) has an embedding for each word in the context.

This embedding is created by adding two distinct embeddings for each input

- token embedding
- positional embedding

Token Embeddings

Embedding matrix E has shape $[|V| \times d]$.

- One row for each of the $|V|$ tokens in the vocabulary.
- Each word is a row vector of d dimensions

Given: string "*Thanks for all the*"

1. Tokenize with BPE and convert into vocab indices

$w = [5, 4000, 10532, 2224]$

2. Select the corresponding rows from E , each row an embedding
- (row 5, row 4000, row 10532, row 2224).



BPE example

Position Embeddings

There are many methods, but we'll just describe the simplest: absolute position.

Goal: learn a position embedding matrix E_{pos} of shape $[1 \times N]$.

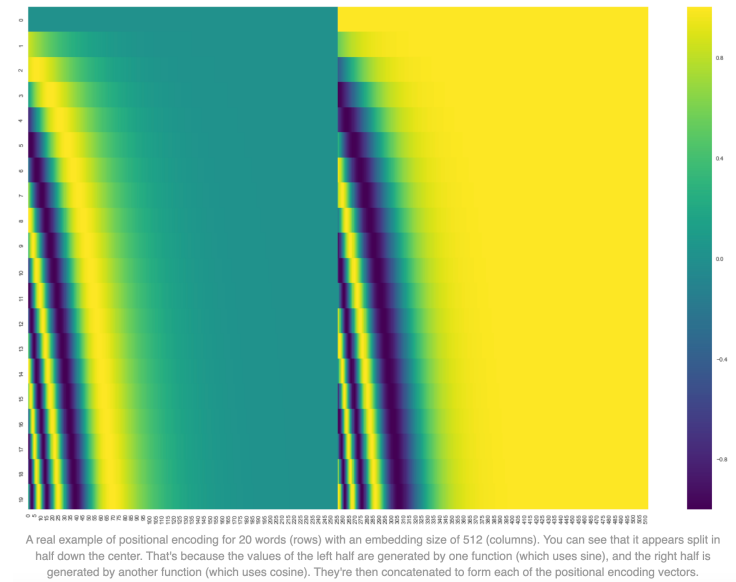
Start with randomly initialized embeddings

- one for each integer up to some maximum length.
- i.e., just as we have an embedding for token *fish*, we'll have an embedding for position 3 and position 17.
- Position embedding can be fixed or learned/trainable
- As with word embeddings, these position embeddings can be learned along with other parameters during training.

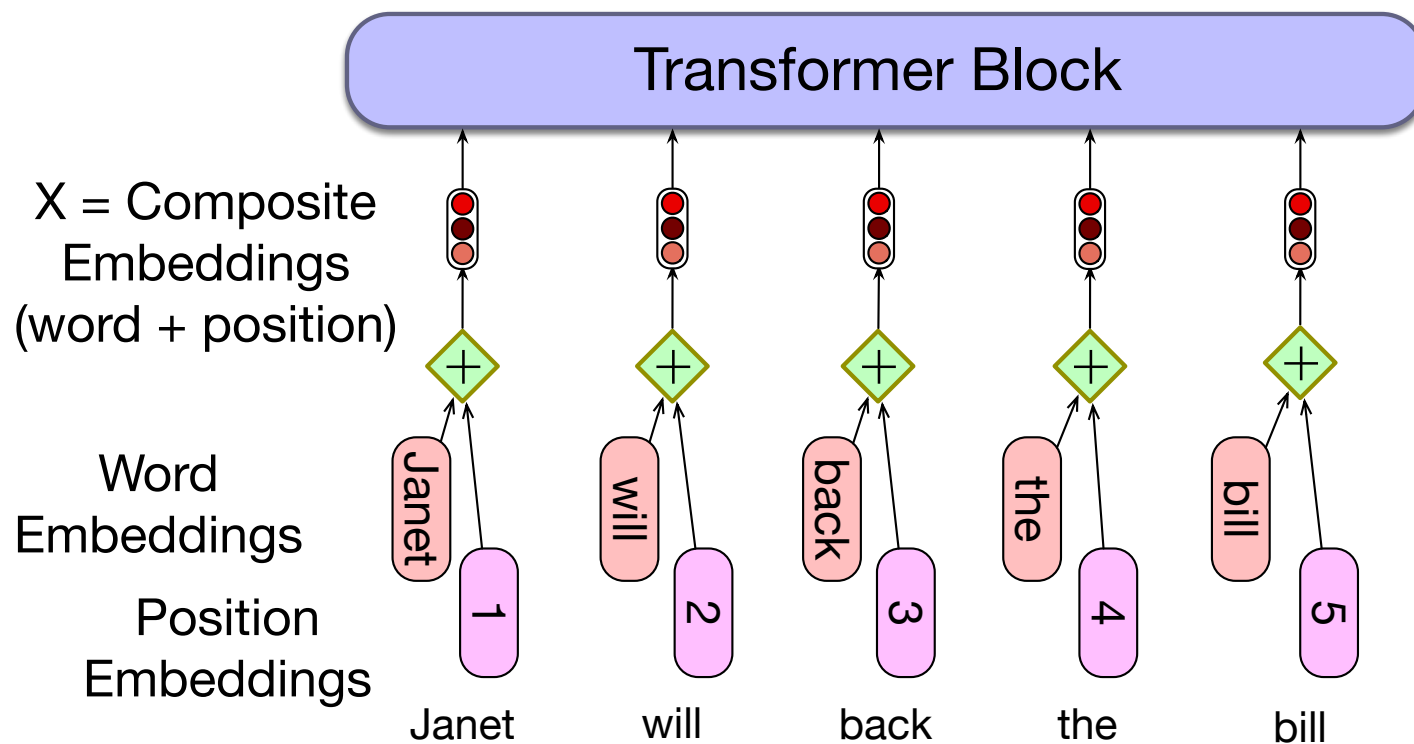
Position embedding

- Self-attention is permutation invariant
- Self-attention is a set operation not sequence
- Without position information in the input embedding
 - Transformer block embedding for "cat sat on mat" will be same as "mat on sat cat"
- Position encoding breaks this symmetry
- Sinusoidal embedding (original Transformer paper)
- Write the equation:

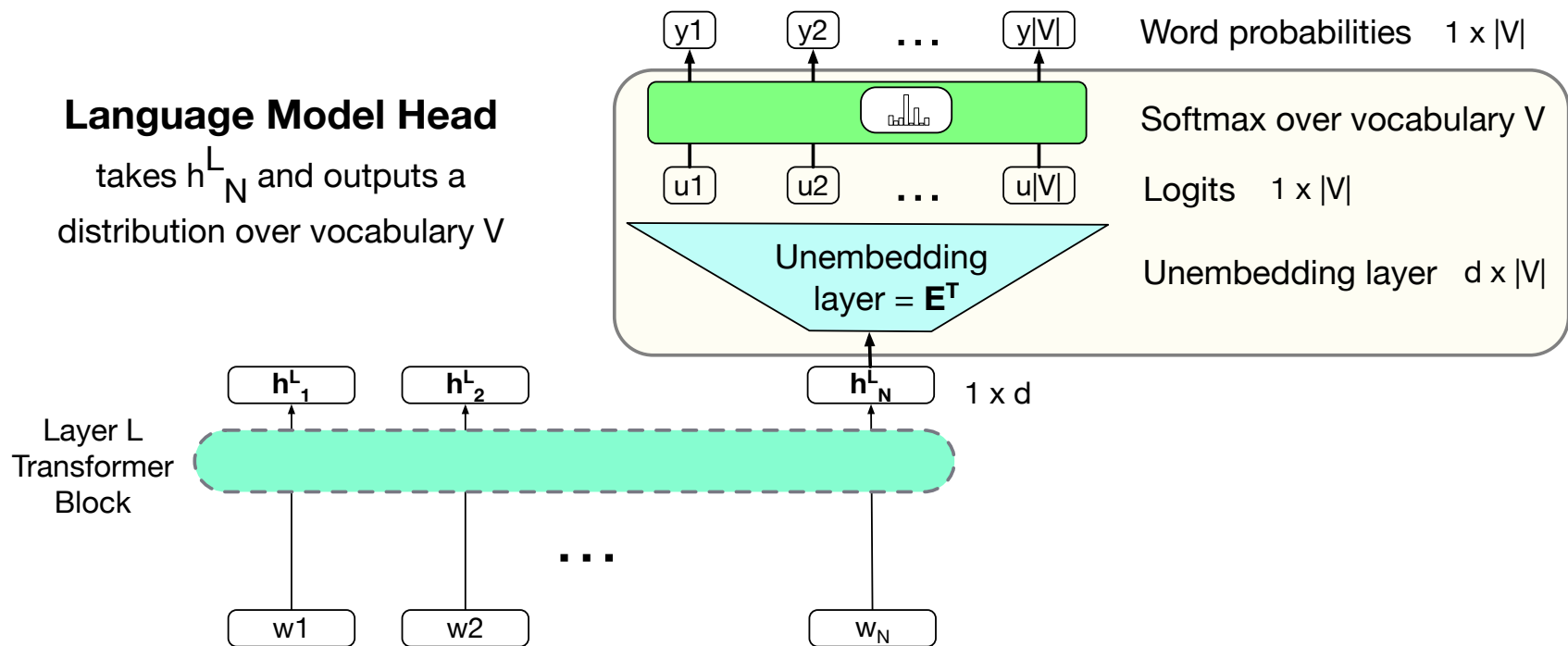
$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$



Each x is just the sum of word and position embeddings

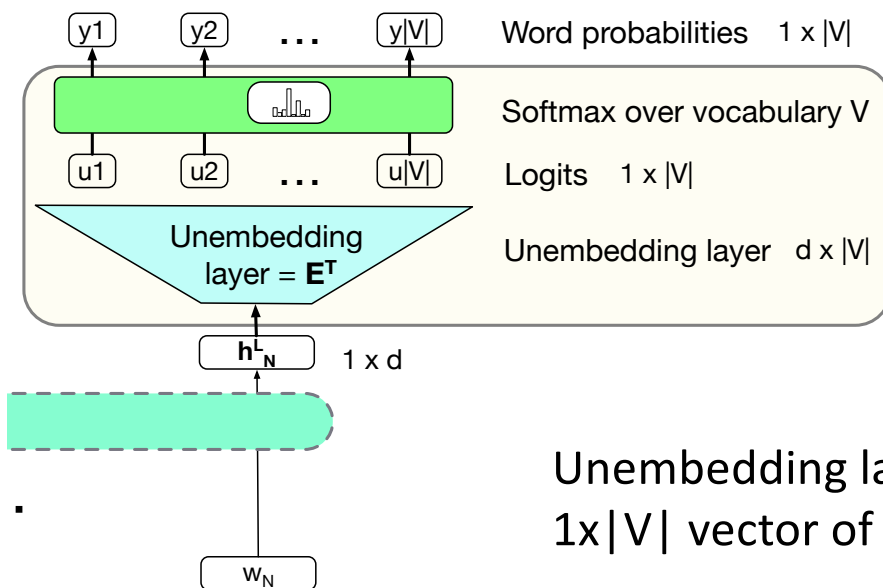


Language modeling head



Language modeling head

Unembedding layer: linear layer projects from h_N^L (shape $[1 \times d]$) to logit vector



Why "unembedding"? **Tied to E^T**

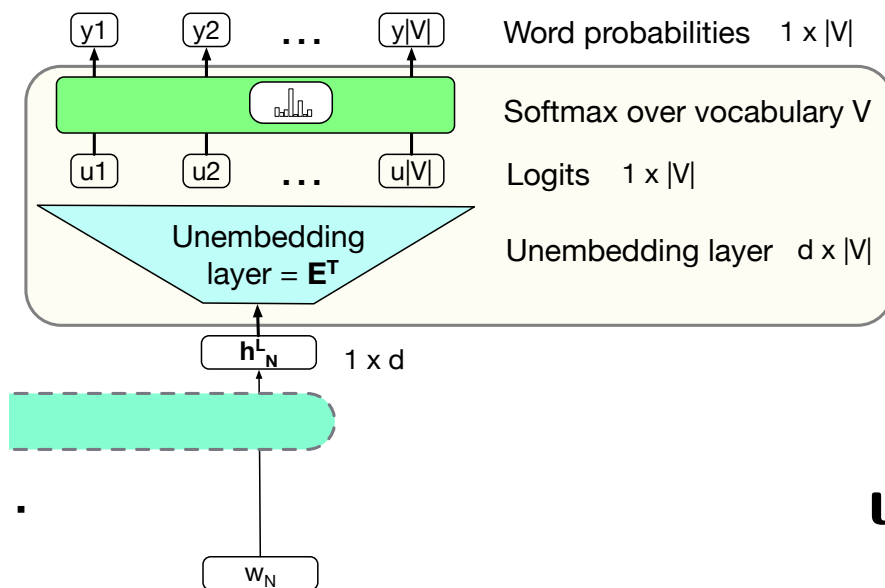
Weight tying, we use the same weights for two different matrices

Unembedding layer maps from an embedding to a $1 \times |V|$ vector of logits

Why weight tying?

- Standard LLM pipeline: Input tokens → Embedding Layer → Transformer Layers → Unembedding Layer → Logits → Softmax
- **Embedding Layer:** Maps token IDs to dense vectors
 $\mathbf{E} \in \mathbb{R}^{(V \times d)}$: vocabulary size V , embedding dimension d
Input: token ID → Output: embedding vector
- **Unembedding Layer:** Maps hidden states to vocabulary logits
 $\mathbf{U} \in \mathbb{R}^{(d \times V)}$: projects d -dimensional hidden states to V logits
Input: hidden vector → Output: vocabulary logits
- Parameter reduction: 50%
- Improved generalization due to regularization

Language modeling head



Logits, the score vector u

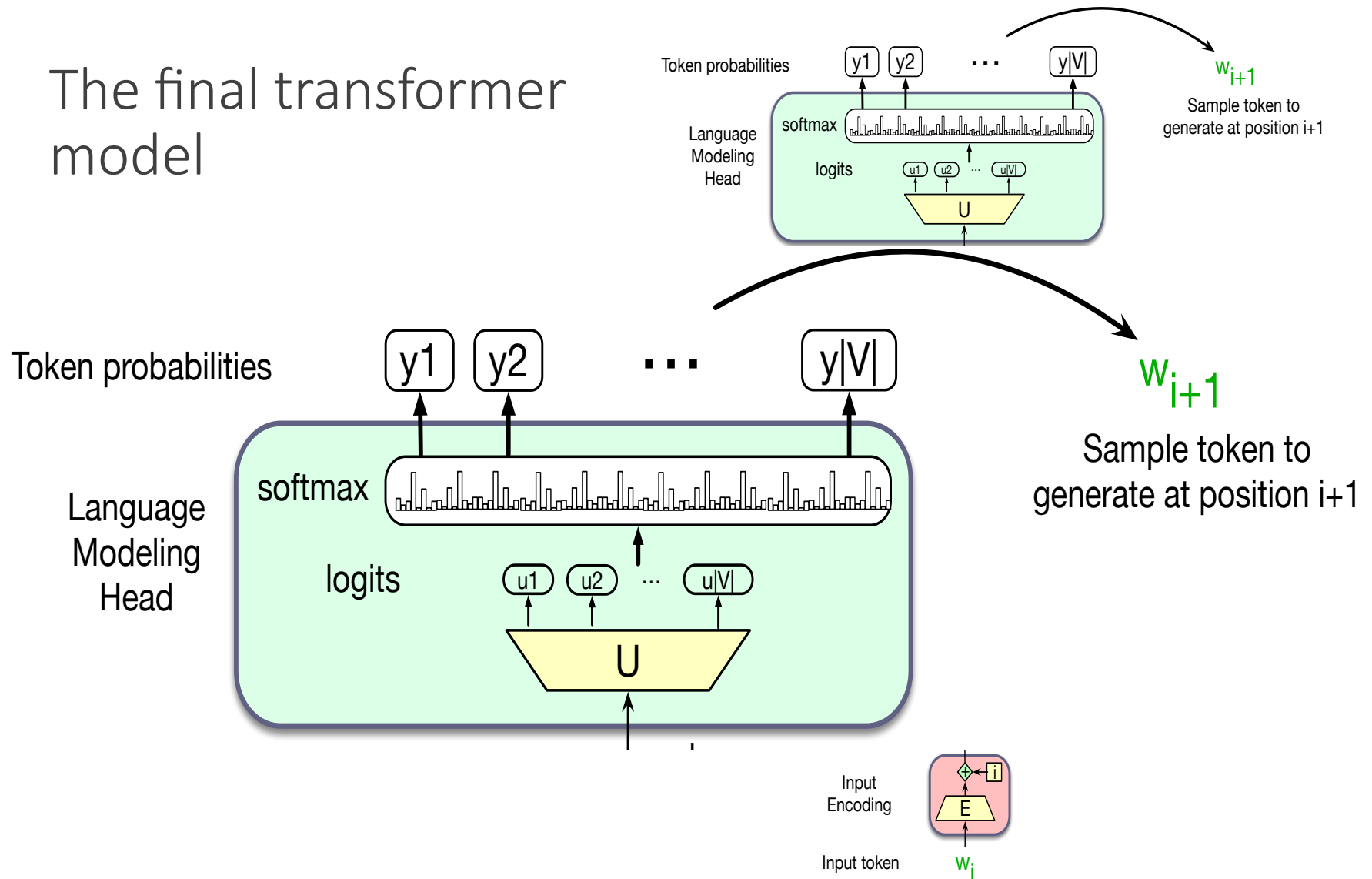
One score for each of the $|V|$ possible words in the vocabulary V .
Shape $1 \times |V|$.

Softmax turns the logits into probabilities over vocabulary.
Shape $1 \times |V|$.

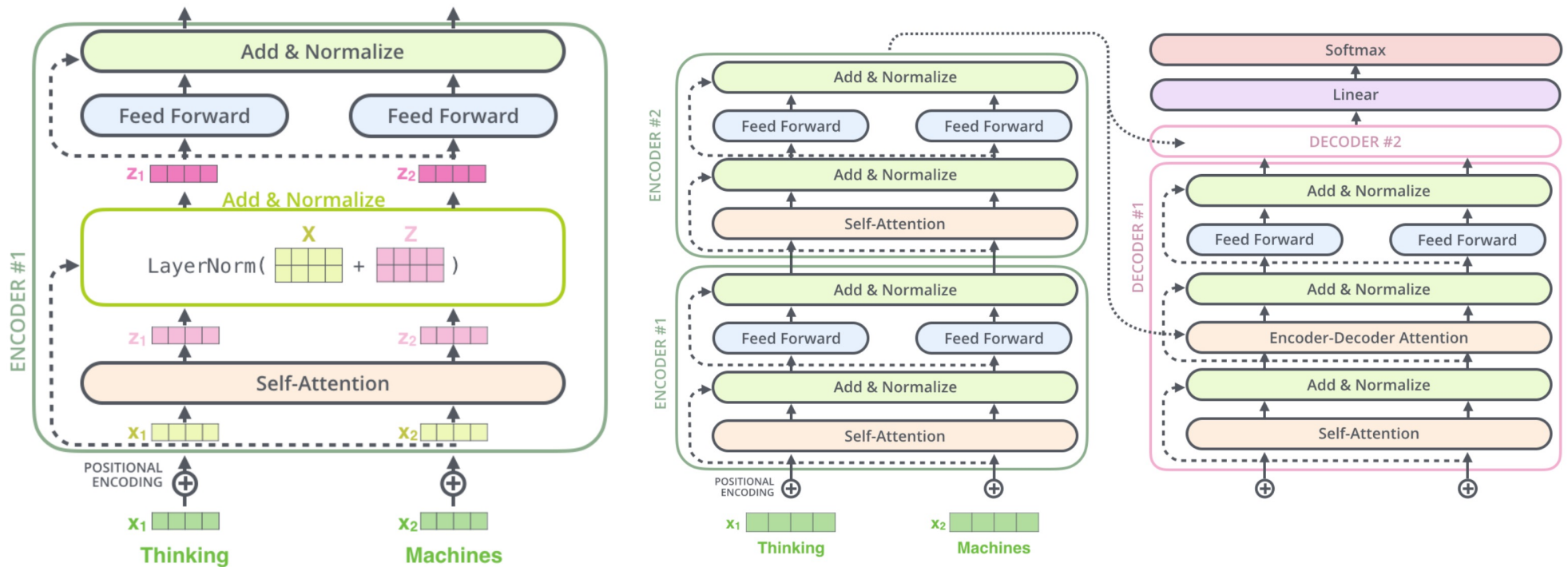
$$\mathbf{u} = \mathbf{h}_N^L \mathbf{E}^T$$

$$\mathbf{y} = \text{softmax}(\mathbf{u})$$

The final transformer model



Residual connection



Layer Norm

- Layer norm is a variation of the z-score from statistics, applied to a single vector in a hidden layer

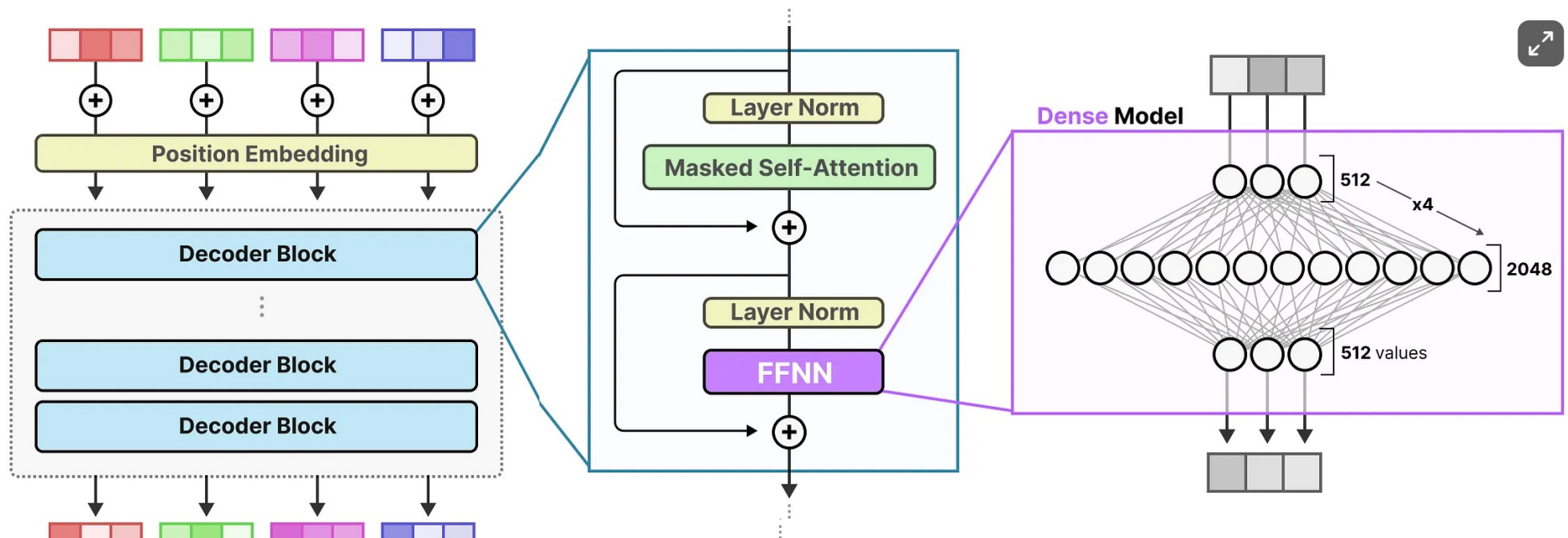
$$\mu = \frac{1}{d} \sum_{i=1}^d x_i$$

$$\sigma = \sqrt{\frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2}$$

$$\hat{\mathbf{x}} = \frac{(\mathbf{x} - \mu)}{\sigma}$$

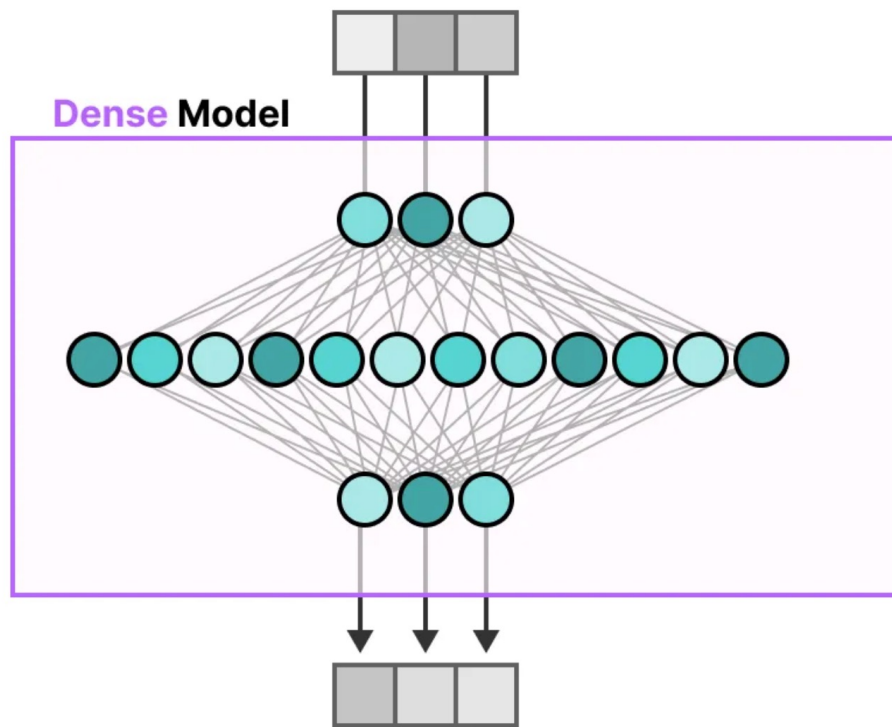
$$\text{LayerNorm}(\mathbf{x}) = \gamma \frac{(\mathbf{x} - \mu)}{\sigma} + \beta$$

FFNN in Decoder

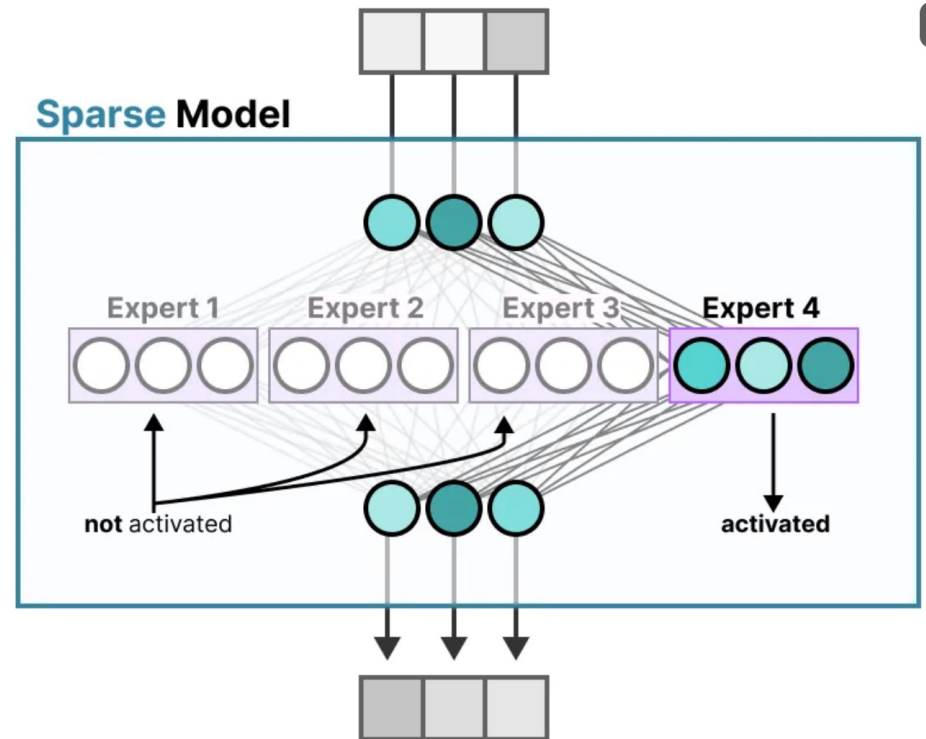


An FFNN allows the model to use the contextual information created by the attention mechanism, transforming it further to capture more complex relationships in the data

Dense vs Sparse Models

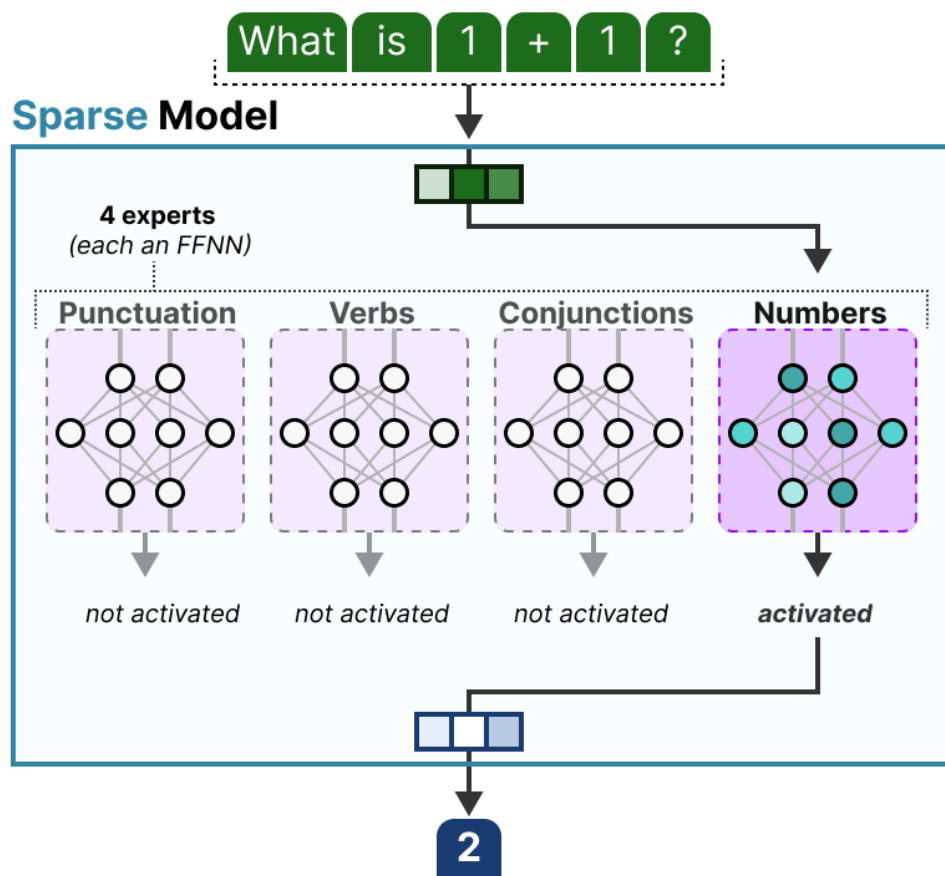


All parameters are activated for any input



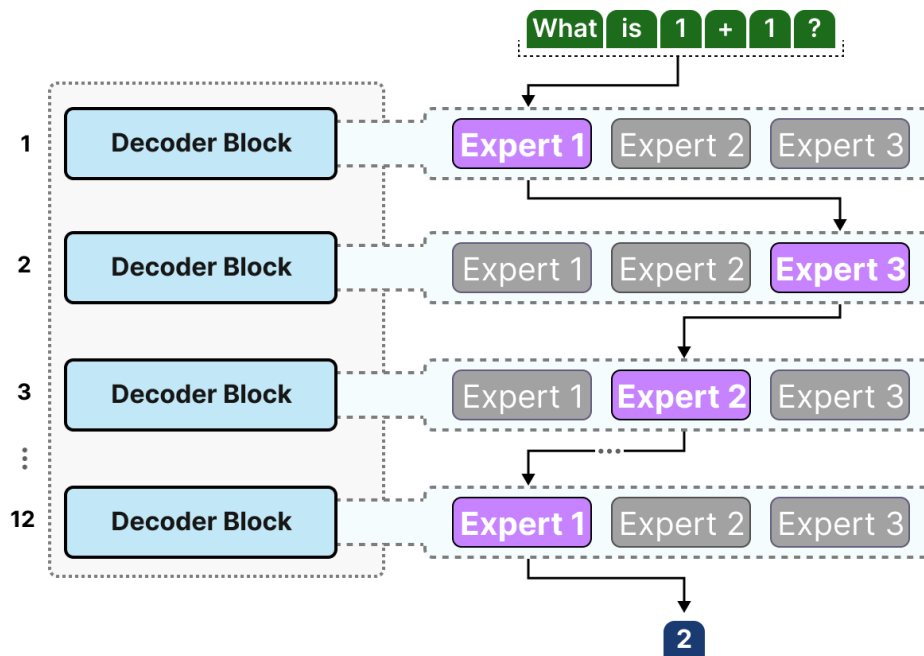
Only a subset of parameters are activated for any input

Architecture of Experts

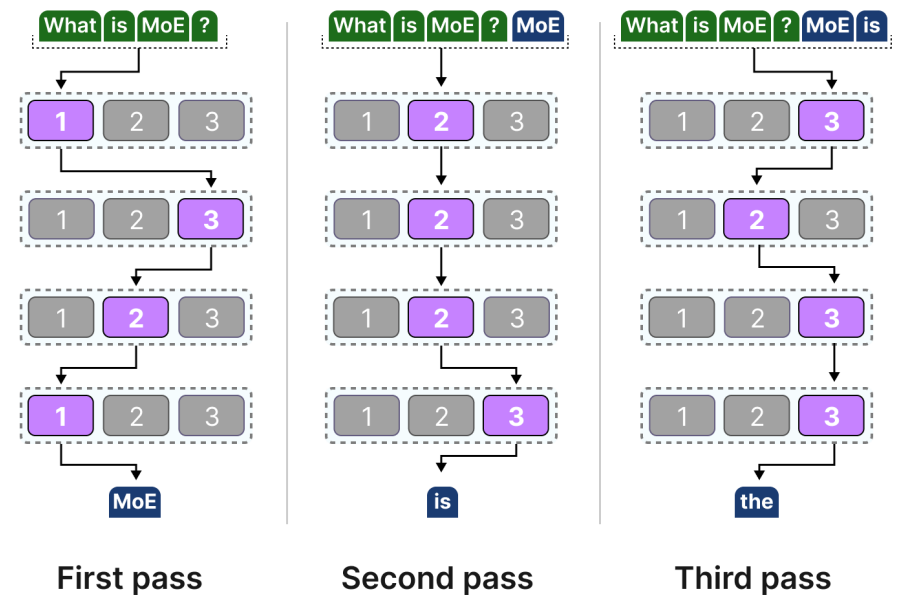


- each expert learns different information during training
- during inference, only specific experts are used as they are most relevant for a given task
- each expert is often whole FFNNs themselves

Choice of Experts

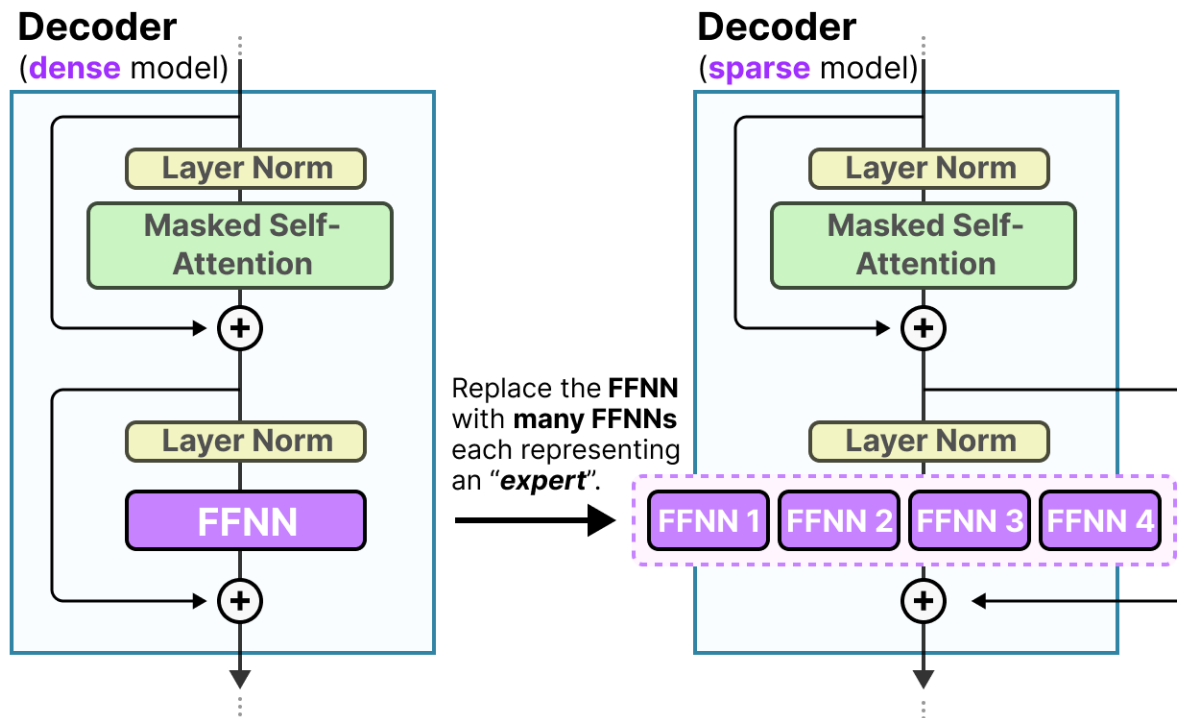


A given text passes through multiple experts before next text is generated

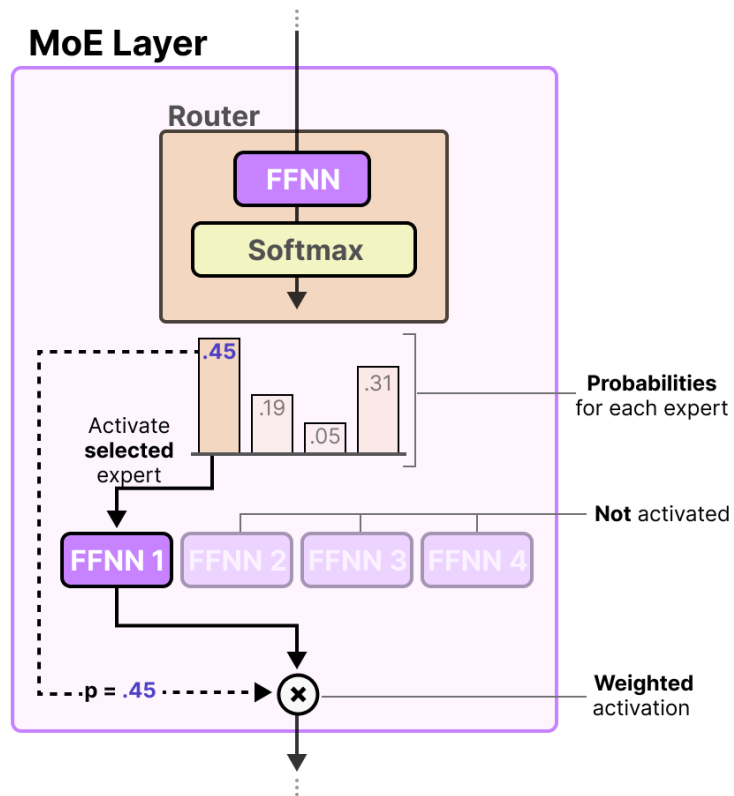


Chosen experts likely differ between inputs which results in different "paths" being taken

Decoder block with experts



Router to select experts



router weights

$$H(\mathbf{x}) = \mathbf{x} * \mathbf{W}$$

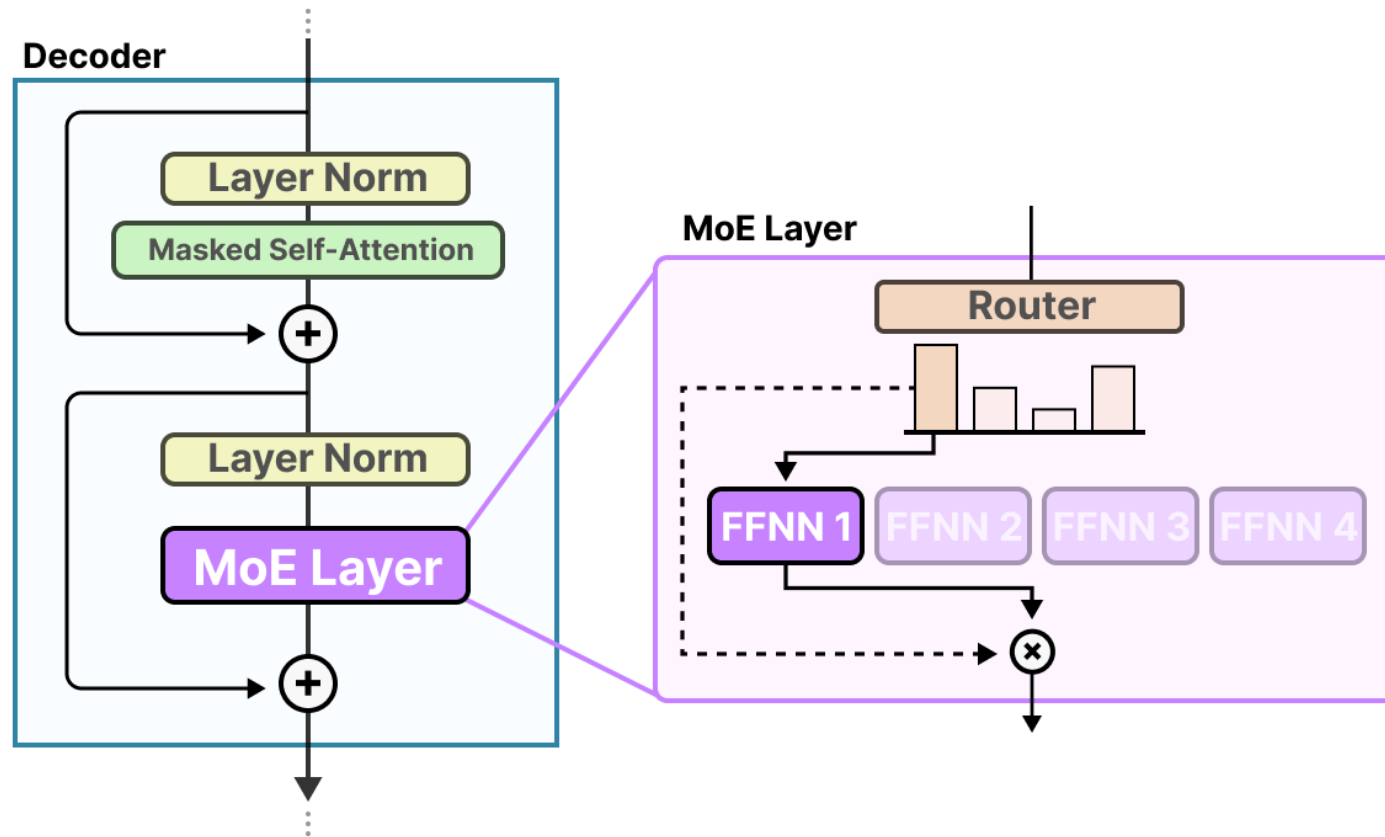
output *input*

probability distribution per expert

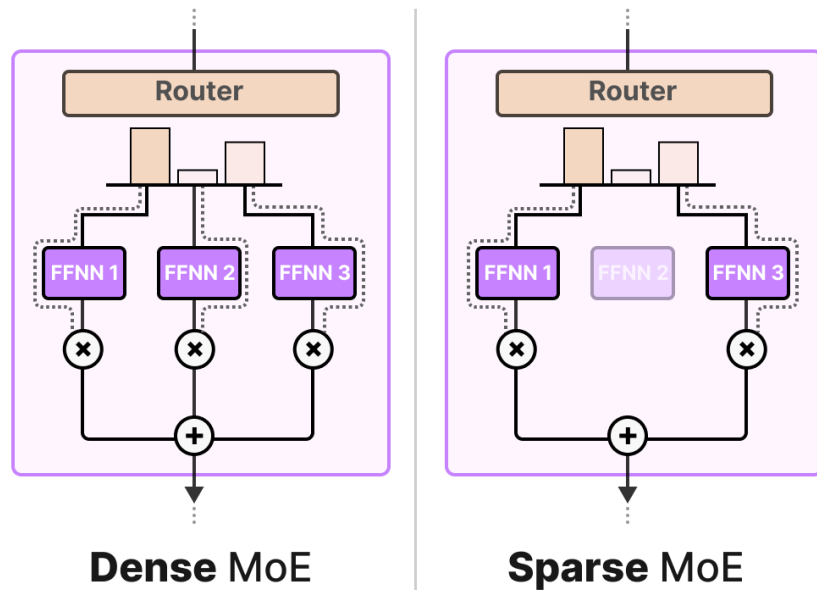
$$G(\mathbf{x}) = \text{Softmax}(H(\mathbf{x}))$$

output

MoE Layer



Sparse vs Dense MoE



Sparse MoE

output

selected *Expert*

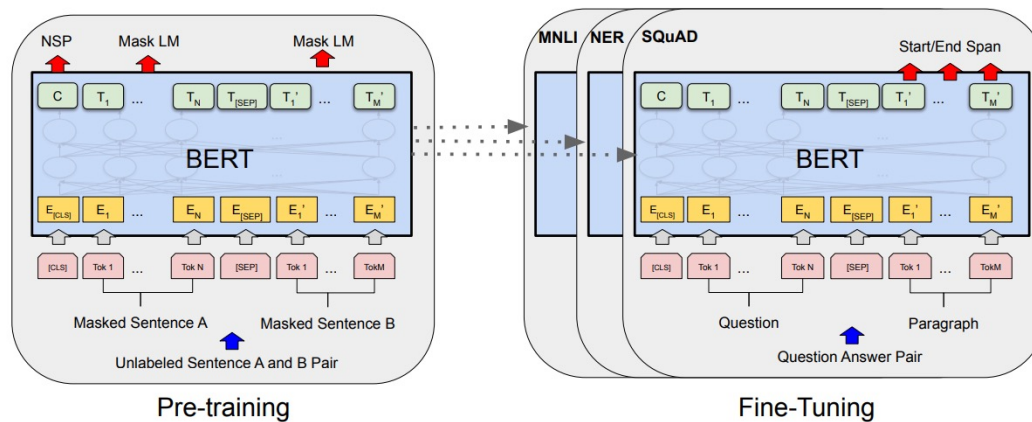
Router Output

Output per *Expert*

$$y = \sum \left(G(x) * E(x) \right)$$

(only 1 expert is chosen in this example)

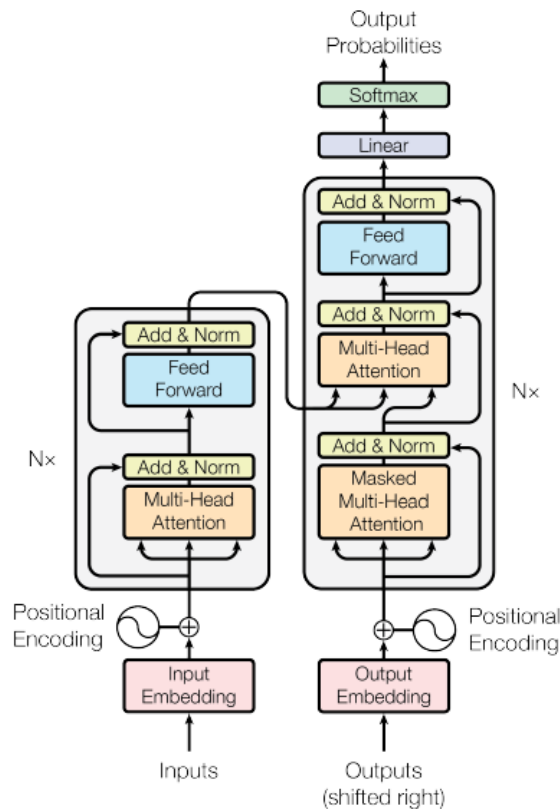
Encoder only LLMs



- Good for: classification, sequence tagging (POS tagging, NER), sentiment analysis
- Examples: BERT, RoBERTa, ALBERT, DeBERTa, etc.
- Typically requires fine-tuning for specific tasks
- Cannot generate text (only understand text)

- Sequence of text → fixed length vector representation for text
- Or sequence of vectors for each text

Encoder-Decoder LLMs (seq2seq)



- Original Transformer (Attention is all you need, 2017)
- Other examples: T5, BART
- Good for: machine translation, summarization
- Image captioning (image is fed into encoder)

Decoder only LLMs: Generative models

- GPT, GPT-2, GPT-3, GPT-4, ChatGPT (OpenAI)
- PaLM (Google)
- Chinchilla (Deepmind)
- LLaMA (Meta)

Transfer Learning in NLP

- Used in standard NLP tasks
 - Question answering
 - Text generation
 - Text summarization
 - Named Entity Recognition
 - Key value pair identification
 - ...
- Transfer learning in NLP through
 - Finetuning language models for down stream tasks
 - Generating contextualized word embeddings

Pre-Training Model Architectures in NLP

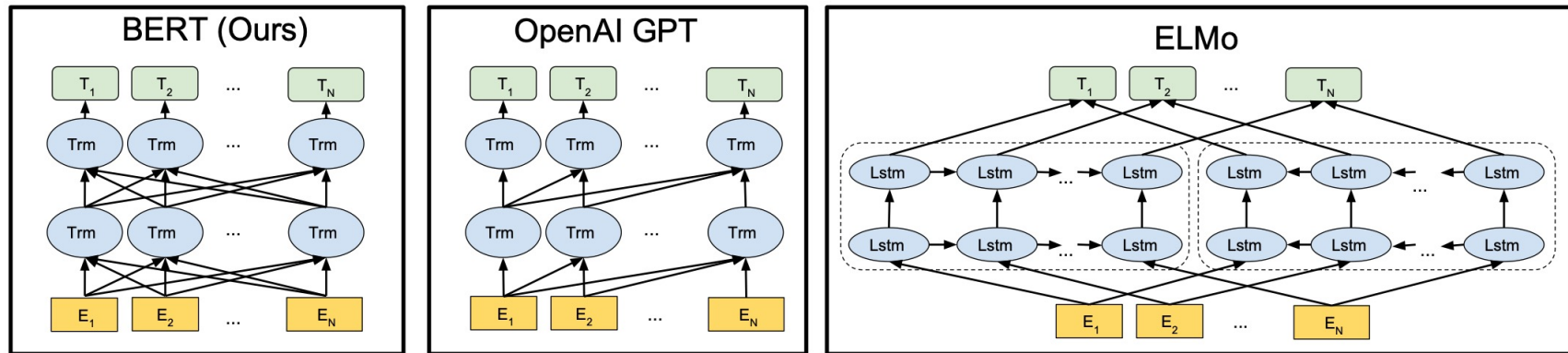


Figure 3: Differences in pre-training model architectures. BERT uses a bidirectional Transformer. OpenAI GPT uses a left-to-right Transformer. ELMo uses the concatenation of independently trained left-to-right and right-to-left LSTMs to generate features for downstream tasks. Among the three, only BERT representations are jointly conditioned on both left and right context in all layers. In addition to the architecture differences, BERT and OpenAI GPT are fine-tuning approaches, while ELMo is a feature-based approach.

Using Transformer for Transfer Learning in NLP

- Transformers handle long-term dependencies better than LSTMs
- Encoder-Decoder structure of the transformer made it perfect for machine translation
- How to pre-train a language model using transformer-based architecture (with attention mechanisms and faster training due to parallelization) that can be finetuned for downstream supervised learning tasks?

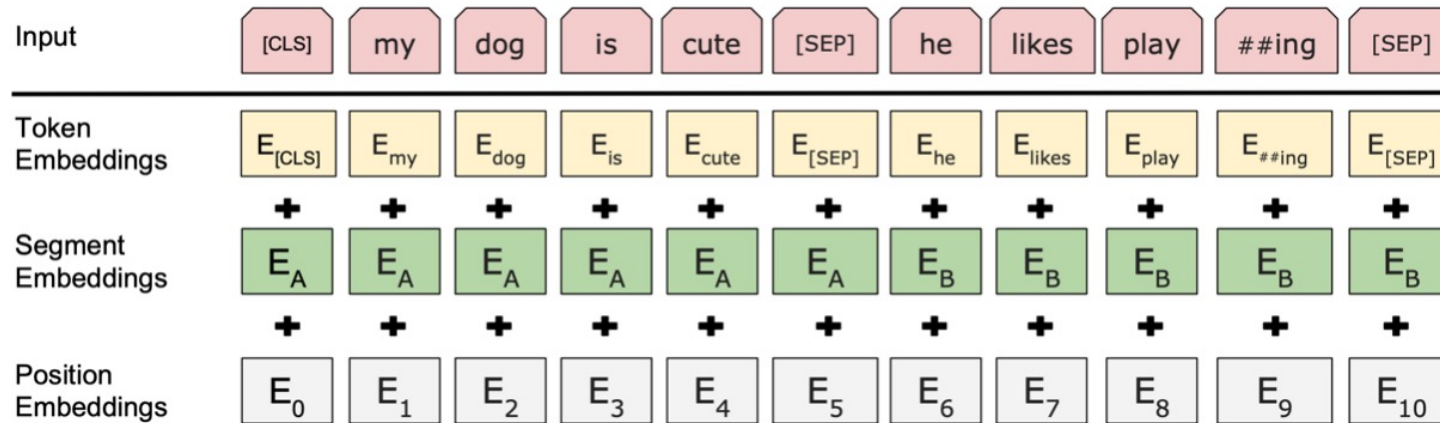
BERT (Bidirectional Encoder Representations from Transformers)

- OpenAI Transformer only trains a forward language model while ELMo's model was bidirectional
- BERT is a transformer-based model whose language model is conditioned on both left and right contexts
- BERT is basically a trained Transformer Encoder stack

BERT Features

- Training corpus was comprised of two entries
 - [Toronto Book Corpus](#) (800M words), and
 - English Wikipedia (2,500M words)
- Two versions of BERT (L stands for the number of layers, H stands for the hidden size, A stands for the number of self-attention heads)
 - BERT-Base: L = 12, H = 768, A = 12, Total parameters = 110M
 - BERT-Large: L = 24, H = 1024, A = 16, Total parameters = 340M

Input Representation of BERT

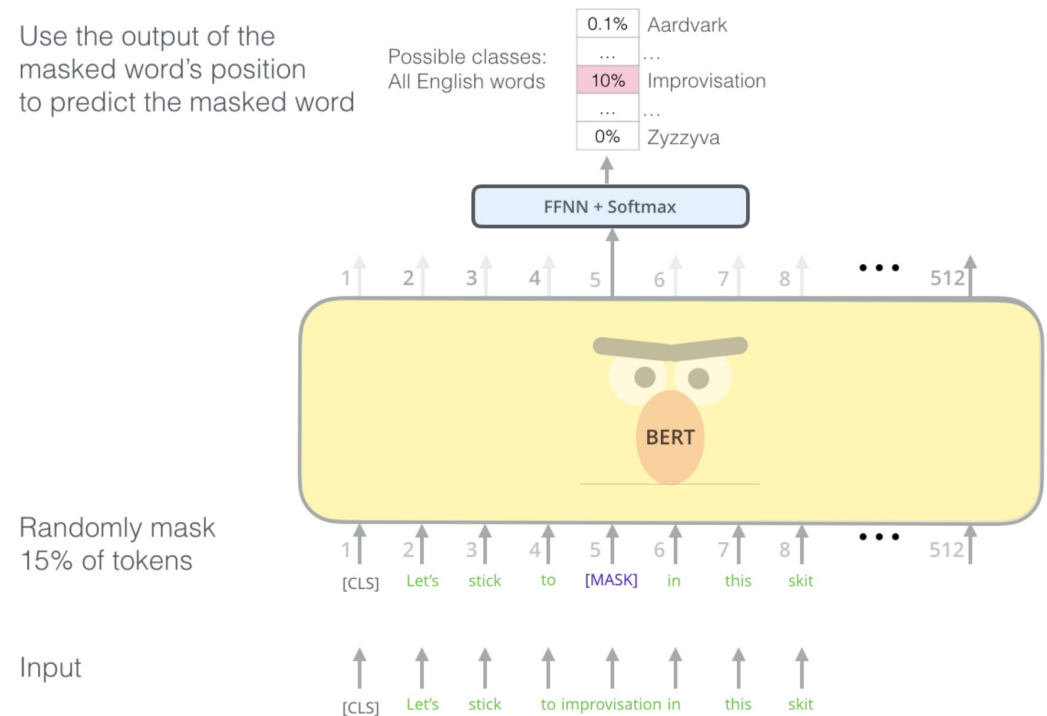


BERT Training – Predicting word at a position

- Masked language model (Masked LM)

That's [mask] she [mask] -> That's what she said

- Randomly masks 15% of tokens in the input and model is trained to predict the masked word



BERT Training – Two sentence tasks

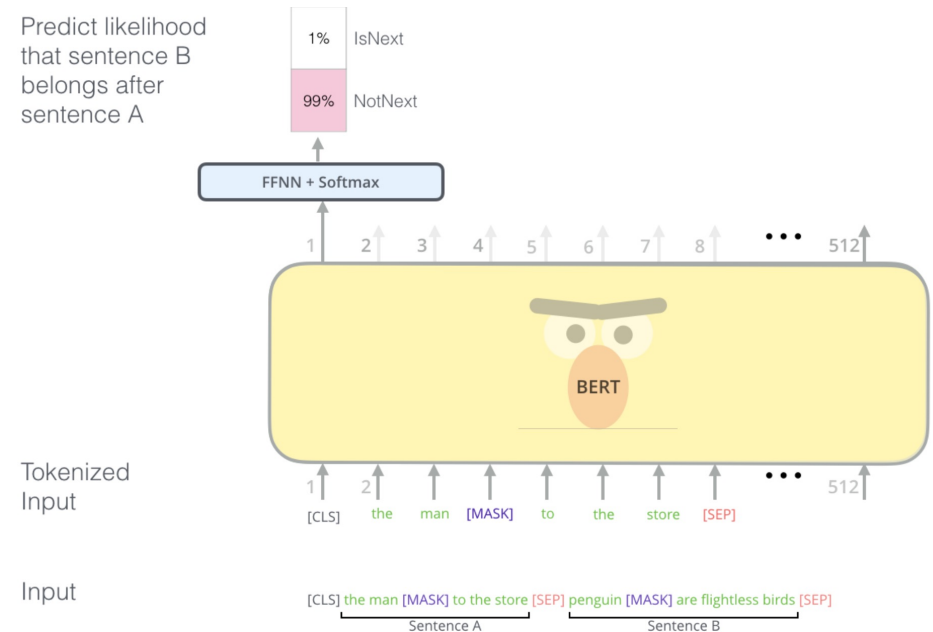
- Next sentence prediction (NSP)
- Task: Given two sentences (A and B), is B likely to be the sentence that follows A, or not? (binary classification)

Input = [CLS] That's [mask] she [mask] . [SEP] Hahaha, nice! [SEP]

Label = *IsNext*

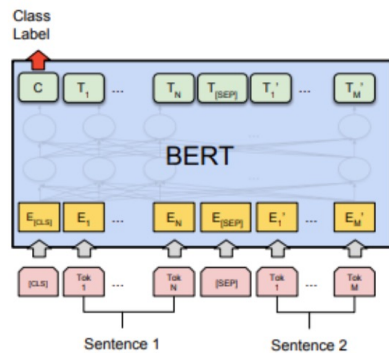
Input = [CLS] That's [mask] she [mask] . [SEP] Dwight, you ignorant [mask] ! [SEP]

Label = *NotNext*

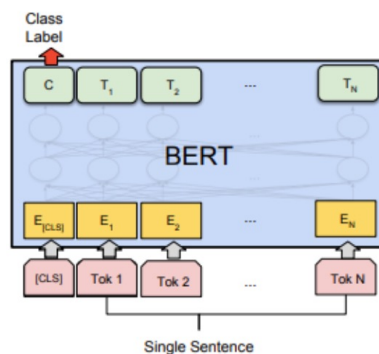


The second task BERT is pre-trained on is a two-sentence classification task. The tokenization is oversimplified in this graphic as BERT actually uses WordPieces as tokens rather than words --- so some words are broken down into smaller chunks.

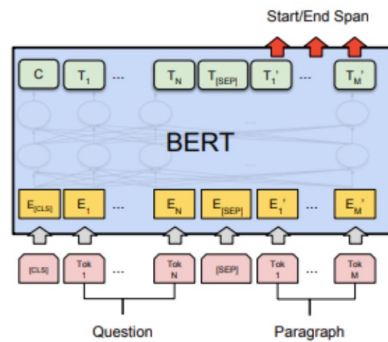
Using BERT for different Downstream Tasks



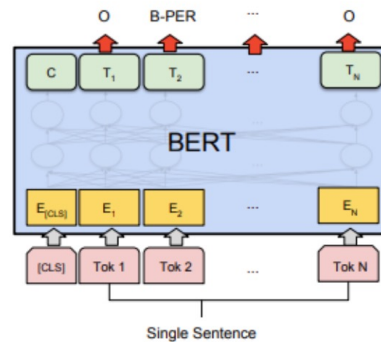
(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



(b) Single Sentence Classification Tasks:
SST-2, CoLA



(c) Question Answering Tasks:
SQuAD v1.1



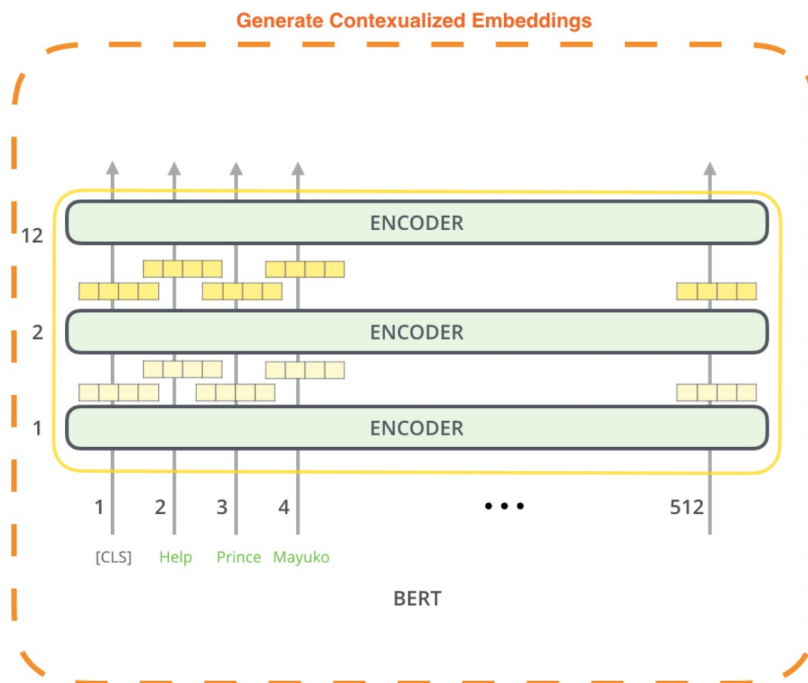
(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

Fine-tuning BERT for Sentiment Analysis

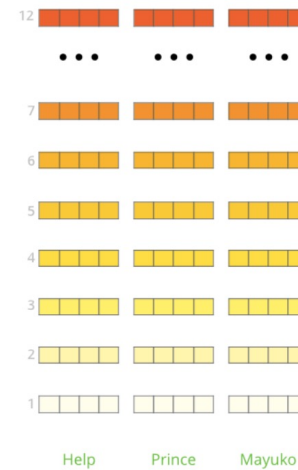
- Sentiment Analysis with BERT [colab tutorial](#)

BERT for Feature Extraction

- Using BERT to create contextualized word embeddings



The output of each encoder layer along each token's path can be used as a feature representing that token.

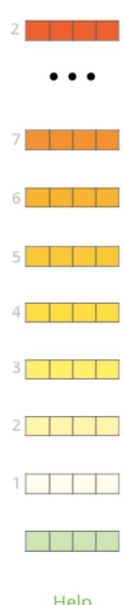
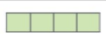
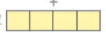
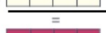
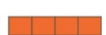
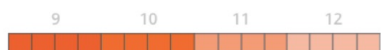


But which one should we use?

- 12 embedding vectors for each word from the output of 12 encoder layers
- Which embedding(s) to use for a given task?

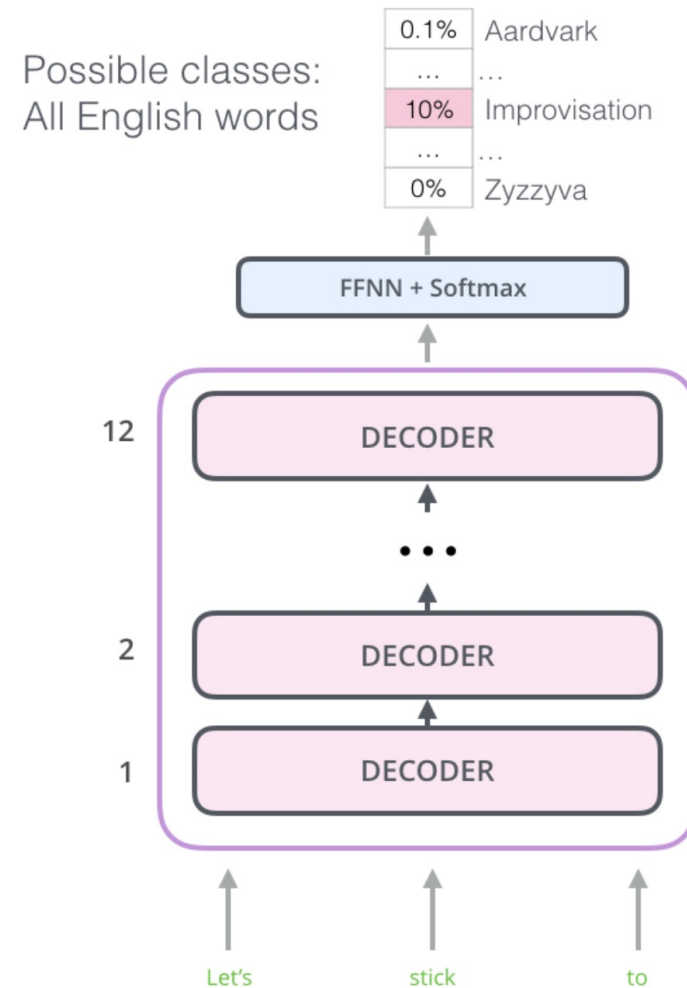
Different ways to create contextualized embeddings for a word using BERT

What is the best contextualized embedding for “Help” in that context?
For named-entity recognition task CoNLL-2003 NER

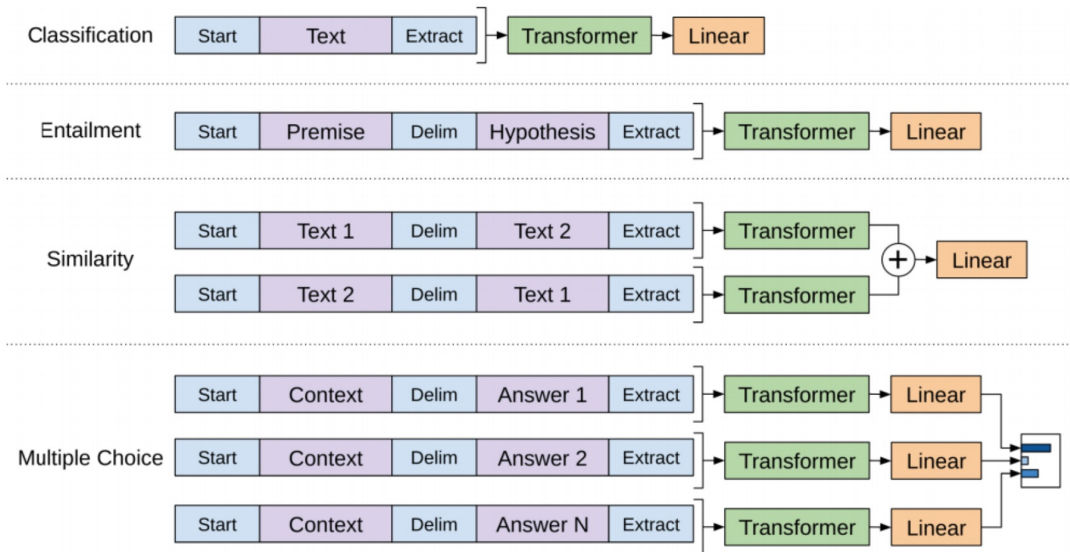
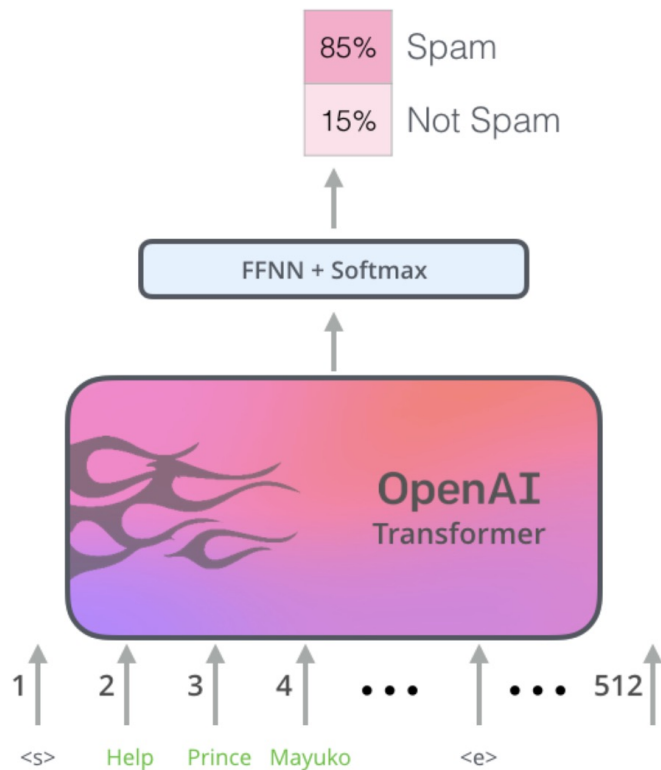
		Dev F1 Score
	First Layer	Embedding  91.0
	Last Hidden Layer	12  94.9
	Sum All 12 Layers	 +  + ... +  +  =  95.5
	Second-to-Last Hidden Layer	11  95.6
	Sum Last Four Hidden	 +  +  +  =  95.9
	Concat Last Four Hidden	 96.1

OpenAI Transformer

- Pre-training a transformer decoder for language modeling
- Decoder masks future tokens so can be trained on next word prediction task
- Model has 12 decoder layers stacked
- Only masked self attention layers



Transfer Learning to Downstream Tasks



GPT-3

- From OpenAI
- Decode only architecture

GPT-3 family of models

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or “GPT-3”	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

GPT-3 training dataset

Dataset	Quantity (tokens)	Weight in training mix	Epochs elapsed when training for 300B tokens
Common Crawl (filtered)	410 billion	60%	0.44
WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4

LLaMa

- From MetaAI
- Decode only architecture

LLaMa family of models

params	dimension	n heads	n layers	learning rate	batch size	n tokens
6.7B	4096	32	32	$3.0e^{-4}$	4M	1.0T
13.0B	5120	40	40	$3.0e^{-4}$	4M	1.0T
32.5B	6656	52	60	$1.5e^{-4}$	4M	1.4T
65.2B	8192	64	80	$1.5e^{-4}$	4M	1.4T

LLaMa training dataset	Dataset	Sampling prop.	Epochs	Disk size
	CommonCrawl	67.0%	1.10	3.3 TB
	C4	15.0%	1.06	783 GB
	Github	4.5%	0.64	328 GB
	Wikipedia	4.5%	2.45	83 GB
	Books	4.5%	2.23	85 GB
	ArXiv	2.5%	1.06	92 GB
	StackExchange	2.0%	1.03	78 GB